

SQL

Note: read sql introduction javatpoint then start this pdf

What is SQL?

SQL stands for Structured Query Language and is a computer language that we use to interact with a relational database. SQL is a tool for *organizing, managing, and retrieving* archived data from a computer database. The original name was given by IBM as Structured English Query Language, abbreviated by the acronym SEQUEL. When data needs to be retrieved from a database, SQL is used to make the request. The [DBMS](#) processes the SQL query retrieves the requested data and returns it to us. Rather, SQL statements describe how a collection of data should be organized or what data should be extracted or added to the database.

Q. what is database?

What is Database?

A database is an organized collection of data, so that it can be easily accessed and managed. The main purpose of the database is to operate a large amount of information by storing, retrieving, and managing data.

What is database schema?

A database schema is a way to logically group objects such as tables, views, stored procedures etc. Think of a schema as a container of objects

Database is technique where we can organize collection of data so that it can be easily accessed and managed, alter the data as well as manipulate the data called as database.

We have n number of technique using that we can manage the database means we can say we have types of database.

Q What is the DBMS?

A Database Management System (DBMS) is software designed to store, retrieve, define, and manage data in a database. DBMS software primarily functions as an interface between the end user and the database, simultaneously managing the data, the database engine, and the database schema in order to facilitate the organization and manipulation of data.

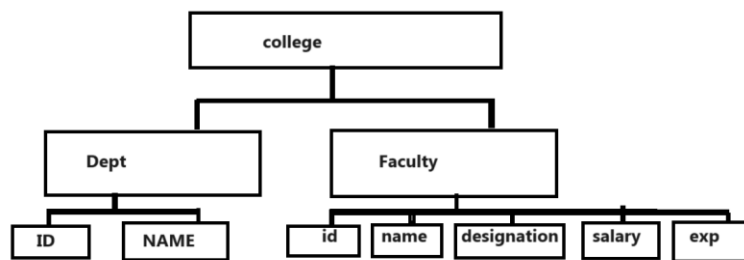
Types of Database (refer javatpoint or notes on website)

1. Relational Database: Relational DBMS is the most widely used DBMS model because it is one of the easiest. This model is based on normalizing data in the rows and columns of the tables. Relational model stored in fixed structures and manipulated using SQL..

Employee

id	name	salary
1	abc	100000
2	mno	50000

2. Hierarchical Database: In a Hierarchical database, model data is organized in a tree-like structure. Data is Stored Hierarchically (top down or bottom up) format. Data is represented using a parent-child relationship. In Hierarchical DBMS parent may have many children, but children have only one parent.



3. Object oriented Database: In Object-oriented Model data stored in the form of objects. The structure which is called classes which display data within it. It is one of the components of DBMS that defines a database as a collection of objects which stores both data member's values and operations.

```

id      = 1
name    = ABC
salary  = 10000
designation = Manager
  
```

Example of object oriented database is mongoDB

4. Network Model: The network database model allows each child to have multiple parents. It helps you to address the need to model more complex relationships like as the orders/parts many-to-many relationship. In this model, entities are organized in a graph which can be accessed through several paths

Now we want to work with Relational Database Management System.

What Is The RDBMS?

A relational database management system (RDBMS or just RDB) is a common type of database that stores data in tables, so it can be used in relation to other stored datasets. Relational Database Management System having the ability to provide the Relationship between Two Different Tables. The data is often stored in many tables, also called 'relations'. These tables are divided into rows, also called records and columns (fields). There can be millions of rows in a database. Columns are made up of one specific data type, like name or price.

The tables in the relational database can be linked in several ways:

- Characteristics of one table record may be linked to a record in another table
- A table record could be linked to many records in another table
- Many table records may be related to many records in another table.

Steps to work with Relational management database system.

1. Install any database tool like as MYSQL, Oracle, and DB2 etc

Now we want to use MYSQL

MYSQL is database tool which help us to provide the environment to work with relational database management system.

<https://dev.mysql.com/downloads/installer/>

Note: You can visit this URL for download the MYSQL

Note: when we install any database tool then we have one default database Example:

Means if we install MYSQL tool then we have one default database in MYSQL name as MYSQL

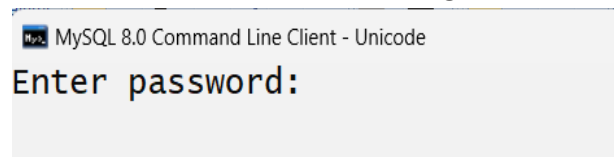
So you can use this default database provided by MYSQL to us.

2. Enter in database: if we want to enter in database or in MYQL tool then we have use following steps.

a) Open MYSQL Tool

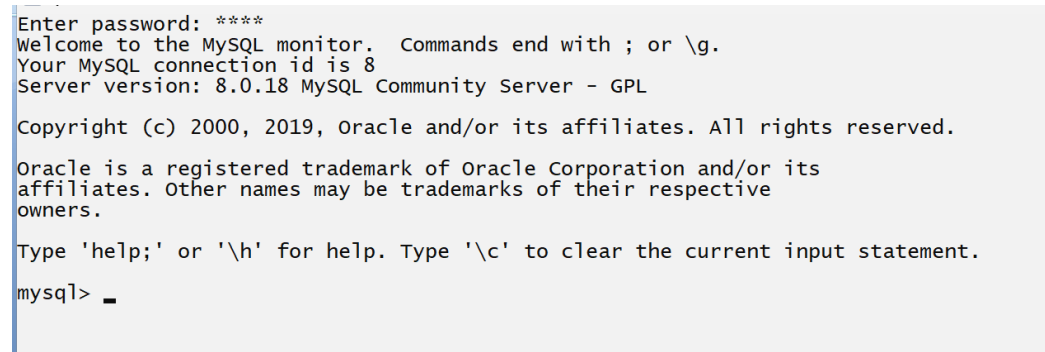
Start menu ---- search menu ---- search mysql --- press enter

After that we have following MYSQL command prompt.



b) Enter the password here which gives at the time of installation of database

Note: here we give the root as password



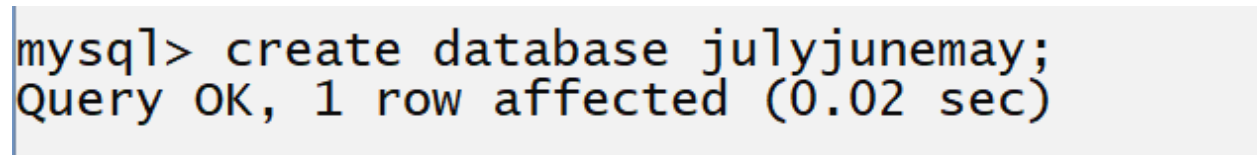
When we input password then we enter in default database name as MYSQL and this database provided by MYSQL tool to us as well as user can create own database also in MYSQL tool.

3. Can create your own database

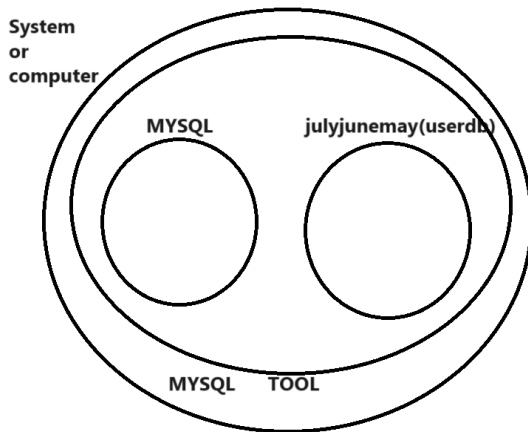
If we want to create your own database we have the following statement

Syntax: create database databasename;

Example: create database julyjunemay



If we execute this command the meaning is we have two databases in mysql one is default database name as mysql and one is user database name as julyjunemay.



Once we create database you can enter in created database and if we want to enter in created database we have statement

Syntax: use databasename;

Example: use julyjunemay

```
mysql> use julyjunemay;  
Database changed  
mysql>
```

The meaning is we enter in database name as julyjunemay

4. Work with Database

If we want to work with relational database we have some types of command provided by relational database to us.

1. DDL: DDL Data definition language means this type of command help us to create table, alter table , drop table etc operation means this type of command work with table structures

2. DML: DML stands for Data Manipulation language which is used for insert data in table, delete record from database table, update record in database table etc.

3. DCL: Data Control Language

4. TCL: Transaction control language

Views	Tables
A view is a virtual table that is extracted from a database.	A table is structured with a set number of columns and a boundless number of rows.
A view does not hold the data itself.	A table contains data and stores it in databases.
A view is utilized to query certain information contained in a few distinct tables.	A table holds fundamental client information and cases of a characterized object.
In a view, we will get frequently queried information.	In a table, changing the information in the database changes the information that appears in the view.

8. What are the subsets of SQL?

SQL queries are divided into four main categories:

- **Data Definition Language (DDL)**

DDL queries are made up of SQL commands that can be used to define the structure of the database and modify it.

- **CREATE:** Used to create databases, tables, indexes, views, and other database objects.
- **DROP:** Used to delete databases, tables, views, and other database objects.
- **ALTER TABLE ... DROP COLUMN:** Used to remove a column from an existing table.
- **ALTER:** Used to modify the structure of an existing table, such as adding, deleting, or modifying columns.
- **TRUNCATE:** Used to remove all records from a table but retains the table's structure for future use.
- **ALTER TABLE ... ADD COLUMN:** Used to add a new column to an existing table.

- **Data Manipulation Language (DML)**

These SQL queries are used to manipulate data in a database.

- **SELECT INTO:** Selects data from one table and inserts it into another
- **INSERT:** Inserts data or records into a table
- **UPDATE:** Updates the value of any record in the database
- **DELETE:** Deletes records from a table
- **Data Control Language (DCL)**
 These SQL queries manage the access rights and permission control of the database.
 - **GRANT:** Grants access rights to database objects
 - **REVOKE:** Withdraws permission from database objects
- **Transaction Control Language (TCL)**
 TCL is a set of commands that essentially manages the transactions in a database and the changes made by the DML statements. TCL allows statements to be grouped together into logical transactions.
 - **COMMIT:** Commits an irreversible transaction, i.e., the previous image of the database before the transaction cannot be retrieved. When a transaction is committed using the COMMIT statement in SQL, it permanently saves the changes made within the transaction to the database. Once committed, the changes cannot be rolled back or undone, and the previous state of the database before the transaction cannot be retrieved without restoring from a backup or utilizing other data recovery methods.
 - **ROLLBACK:** Reverts the steps in a transaction in case of an error
 - **SAVEPOINT:** Sets a savepoint in the transaction to which rollback can be executed
 - **SET TRANSACTION:** Sets the characteristics of the transaction

9. What are the applications of SQL?

The major applications of SQL are listed below:

- Writing data integration scripts
- Setting and running analytical queries
- Retrieving subsets of information within a database for analytics applications and transaction processing

- Adding, updating, and deleting rows and columns of data in a database

SQL Data Types

The data type of a column defines what value the column can hold: integer, character, money, date and time, binary, and so on.

Each column in a database table is required to have a name and a data type.

An SQL developer must decide what type of data that will be stored inside each column when creating a table. The data type is a guideline for SQL to understand what type of data is expected inside of each column, and it also identifies how SQL will interact with the stored data.

MySQL Data Types (Version 8.0):

In MySQL there are three main data types: **string, numeric, and date and time.**

String Data Types

Data type	Description
CHAR(size)	A FIXED length string (can contain letters, numbers, and special characters). The size parameter specifies the column length in characters - can be from 0 to 255. Default is 1
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The size parameter specifies the maximum string length in characters - can be from 0 to 65535
BINARY(size)	Equal to CHAR(), but stores binary byte strings. The size parameter specifies the column length in bytes. Default is 1
VARBINARY(size)	Equal to VARCHAR(), but stores binary byte strings. The size parameter specifies the maximum column length in bytes.
TINYBLOB	For BLOBs (Binary Large Objects). Max length: 255 bytes
TINYTEXT	Holds a string with a maximum length of 255 characters
TEXT(size)	Holds a string with a maximum length of 65,535 bytes
BLOB(size)	For BLOBs (Binary Large Objects). Holds up to 65,535 bytes of data
MEDIUMTEXT	Holds a string with a maximum length of 16,777,215 characters
MEDIUMBLOB	For BLOBs (Binary Large Objects). Holds up to 16,777,215 bytes of data
LONGTEXT	Holds a string with a maximum length of 4,294,967,295 characters
LOBLOB	For BLOBs (Binary Large Objects). Holds up to 4,294,967,295 bytes of data
ENUM(val1, val2, val3, ...)	A string object that can have only one value, chosen from a list of possible values. You can list up to 65535 values in an ENUM list. If a value is inserted that is not in the list, a blank value will be inserted. The values are sorted in the order you enter them
SET(val1, val2, val3, ...)	A string object that can have 0 or more values, chosen from a list of possible values. You can list up to 64 values in a SET list

Numeric Data Types

Data type	Description
BIT(<i>size</i>)	A bit-value type. The number of bits per value is specified in <i>size</i> . The <i>size</i> parameter can hold a value from 1 to 64. The default value for <i>size</i> is 1.
TINYINT(<i>size</i>)	A very small integer. Signed range is from -128 to 127. Unsigned range is from 0 to 255. The <i>size</i> parameter specifies the maximum display width (which is 255)
BOOL	Zero is considered as false, nonzero values are considered as true.
BOOLEAN	Equal to BOOL
SMALLINT(<i>size</i>)	A small integer. Signed range is from -32768 to 32767. Unsigned range is from 0 to 65535. The <i>size</i> parameter specifies the maximum display width (which is 255)
MEDIUMINT(<i>size</i>)	A medium integer. Signed range is from -8388608 to 8388607. Unsigned range is from 0 to 16777215. The <i>size</i> parameter specifies the maximum display width (which is 255)
INT(<i>size</i>)	A medium integer. Signed range is from -2147483648 to 2147483647. Unsigned range is from 0 to 4294967295. The <i>size</i> parameter specifies the maximum display width (which is 255)
INTEGER(<i>size</i>)	Equal to INT(<i>size</i>)
BIGINT(<i>size</i>)	A large integer. Signed range is from -9223372036854775808 to 9223372036854775807. Unsigned range is from 0 to 18446744073709551615. The <i>size</i> parameter specifies the maximum display width (which is 255)
FLOAT(<i>size</i> , <i>d</i>)	A floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. This syntax is deprecated in MySQL 8.0.17, and it will be removed in future MySQL versions
FLOAT(<i>p</i>)	A floating point number. MySQL uses the <i>p</i> value to determine whether to use FLOAT or DOUBLE for the resulting data type. If <i>p</i> is from 0 to 24, the data type becomes FLOAT(). If <i>p</i> is from 25 to 53, the data type becomes DOUBLE()
DOUBLE(<i>size</i> , <i>d</i>)	A normal-size floating point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter
DOUBLE PRECISION(<i>size</i> , <i>d</i>)	
DECIMAL(<i>size</i> , <i>d</i>)	An exact fixed-point number. The total number of digits is specified in <i>size</i> . The number of digits after the decimal point is specified in the <i>d</i> parameter. The maximum number for <i>size</i> is 65. The maximum number for <i>d</i> is 30. The default value for <i>size</i> is 10. The default value for <i>d</i> is 0.
DEC(<i>size</i> , <i>d</i>)	Equal to DECIMAL(<i>size</i> , <i>d</i>)

Note: All the numeric data types may have an extra option: UNSIGNED or ZEROFILL. If you add the UNSIGNED option, MySQL disallows negative values for the column. If you add the ZEROFILL option, MySQL automatically also adds the UNSIGNED attribute to the column.

Date and Time Data Types

Data type	Description
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME(<i>fsp</i>)	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time
TIMESTAMP(<i>fsp</i>)	A timestamp. TIMESTAMP values are stored as the number of seconds since the Unix epoch ('1970-01-01 00:00:00' UTC). Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1970-01-01 00:00:01' UTC to '2038-01-09 03:14:07' UTC. Automatic initialization and updating to the current date and time can be specified using DEFAULT CURRENT_TIMESTAMP and ON UPDATE CURRENT_TIMESTAMP in the column definition
TIME(<i>fsp</i>)	A time. Format: hh:mm:ss. The supported range is from '-838:59:59' to '838:59:59'
YEAR	A year in four-digit format. Values allowed in four-digit format: 1901 to 2155, and 0000. MySQL 8.0 does not support year in two-digit format.

Now want to discuss about the DDL Command

DDL contain following types of command

a) Create b) alter c) desc d) drop e) truncate

Create Command

Create Command help us to create the table, create the database, create view, create procedure, create trigger etc

How to create table by using create command

Syntax: create table tablename(columnname datatype(size),column name datatype(size));

Example:

```
mysql> use julyjunemay;
Database changed
mysql> create table employee(id int(5),name varchar(200),email varchar(200),sal int(5));
Query OK, 0 rows affected, 2 warnings (0.15 sec)

mysql>
```

Once we create table and if we want to check table description means column name, data type, size etc we have command name as desc i.e. describe

Syntax: desc tablename;

Example: desc employee;

```
mysql> desc employee;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(5)        | YES  |     | NULL    |       |
| name  | varchar(200)  | YES  |     | NULL    |       |
| email | varchar(200)  | YES  |     | NULL    |       |
| sal   | int(5)        | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.06 sec)

mysql> _
```

Alter command

Alter command is used for modify the table structure means using alter command we can add new column in table after creation , drop column from table , rename column , and modify the column

There are four options in Alter Statement

1. Add New Column in Table: if we think about alter statement we can add new column in table after creating

Syntax: alter table tablename add column (columnname datatype (size));

or

alter table tablename add column(columnname datatype(size), columnname datatype(size));

```
mysql> use july2023;
Database changed
mysql> create table employee(id int(5),name varchar(200),sal int(5));
Query OK, 0 rows affected, 2 warnings (0.08 sec)

mysql> commit;
Query OK, 0 rows affected (0.02 sec)

mysql>
```

If we think about above table we have three column in employee table id, name and sal and we want to add new columns in table email and address so we can use alter command with employee table given below.

```
mysql> alter table employee add column(email varchar(200),address varchar(200));
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	YES		NULL	
name	varchar(200)	YES		NULL	
sal	int(5)	YES		NULL	
email	varchar(200)	YES		NULL	
address	varchar(200)	YES		NULL	

5 rows in set (0.09 sec)

after alter we have two extra columns in employee table.

2. Drop Column from Table: if we want to drop any column from table we have to use drop in alter command

Syntax: alter table tablename drop column column name;

If we think about above table we have employee table with column id, name, Sal, email and address But we want to drop the address column from employee table so alter statement look like as.

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	YES		NULL	
name	varchar(200)	YES		NULL	
sal	int(5)	YES		NULL	
email	varchar(200)	YES		NULL	
address	varchar(200)	YES		NULL	

5 rows in set (0.09 sec)

```
mysql> alter table employee drop column address;
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	YES		NULL	
name	varchar(200)	YES		NULL	
sal	int(5)	YES		NULL	
email	varchar(200)	YES		NULL	

4 rows in set (0.00 sec)

before drop column table

after drop column

3. Rename column name: if we want to change column name after table creation we have rename column name option available in alter command.

Syntax: alter table tablename rename column oldcolumn name to new columnname;

Example:

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	YES		NULL	
name	varchar(200)	YES		NULL	
sal	int(5)	YES		NULL	
email	varchar(200)	YES		NULL	

4 rows in set (0.00 sec)

If we think about this table we have column name as sal and we want to rename this column from sal to salary then your alter command look like as.

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	YES		NULL	
name	varchar(200)	YES		NULL	
sal	int(5)	YES		NULL	
email	varchar(200)	YES		NULL	

4 rows in set (0.00 sec)

before rename column

```
mysql> alter table employee rename column sal to salary;  
Query OK, 0 rows affected (0.04 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	YES		NULL	
name	varchar(200)	YES		NULL	
salary	int(5)	YES		NULL	
email	varchar(200)	YES		NULL	

4 rows in set (0.00 sec)

after rename column

4. Modify the column data type or size

If we want to modify the column data type and size using alter command we have modify column option in alter statement

Syntax: alter table tablename modify column columnname datatype(size);

If we think about above screen short we have employee table with email column whose size is 200 and we want to change the size of email column from 200 to 20 then your alter command or statement look like as

```
alter table employee modify email varchar(20);
```

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	YES		NULL	
name	varchar(200)	YES		NULL	
salary	int(5)	YES		NULL	
email	varchar(200)	YES		NULL	

4 rows in set (0.00 sec)

mysql> alter table employee modify email varchar(20);
Query OK, 0 rows affected (0.15 sec)
Records: 0 Duplicates: 0 Warnings: 0

```
mysql> desc employee;
```

Field	Type	Null	Key	Default	Extra
id	int(5)	YES		NULL	
name	varchar(200)	YES		NULL	
salary	int(5)	YES		NULL	
email	varchar(20)	YES		NULL	

4 rows in set (0.00 sec)

before modification

after modification

Drop Command: this statement help us to drop or delete the table or delete view or delete the procedure or delete the trigger etc.

Syntax: drop table tablename;

```
mysql> drop table employee;
Query OK, 0 rows affected (0.06 sec)

mysql> desc employee;
ERROR 1146 (42S02): Table 'july2023.employee' doesn't exist
mysql> _
```

Note: above desc command generate error to us employee does not exist because we remove or delete the table

DML statements or Command

There are four types of DML statements

1. Insert: this statement help us to insert the data or record in database table.

There are two types of insert Statement

a) Wild card inserts: wild card insert means we insert the value in all column of table

Syntax: insert into tablename values ('value1', 'value2', 'value....n');

Example: suppose consider we have employee table with four column id, name, sal and email and we want to insert data in employee table.

```
mysql> create table employee(id int(5),name varchar(200),sal int(5),email varchar(200));
Query OK, 0 rows affected, 2 warnings (0.07 sec)
```

```
mysql> insert into employee values(1,'ram',10000,'ram@gmail.com');
Query OK, 1 row affected (0.03 sec)
```

```
mysql> insert into employee values(2,'shyam',20000,'shyam@gmail.com');
Query OK, 1 row affected (0.03 sec)
```

```
mysql> select *from employee;
```

id	name	sal	email
1	ram	10000	ram@gmail.com
2	shyam	20000	shyam@gmail.com

2 rows in set (0.00 sec)

Note: if we think about above table we have four column id,name ,sal and email and we insert values in four column

b) Partial insert: partial insert means we insert the data in specified column in row called as partial selection

Suppose consider we have table name as employee with four column id, name, sal and email and we want to insert only id and name of employee.

Syntax: insert into tablename (column1, column2....column...n)values(value1,value2....value...n)

In this syntax we use column name before values which we want to insert in database table

Example: we want to insert only id and name of employee

Syntax: insert into employee (id, name) values (1,'Ganesh');

```
mysql> insert into employee (id,name) values(1,'Ganesh');
Query OK, 1 row affected (0.03 sec)
```

```
mysql> select *from employee;
```

id	name	sal	email
1	ram	10000	ram@gmail.com
2	shyam	20000	shyam@gmail.com
1	Ganesh	NULL	NULL

3 rows in set (0.00 sec)

Note: if we think about above insert statement we insert only id and name of employee and we not insert sal and email so the sal and email value is null inserted by default by database.

What is a NULL Value?

A field with a NULL value is a field with no value.

If a field in a table is optional, it is possible to insert a new record or update a record without adding a value to this field. Then, the field will be saved with a NULL value

Note: A NULL value is different from a zero value or a field that contains spaces. A field with a NULL value is one that has been left blank during record creation!

How to Test for NULL Values?

It is not possible to test for NULL values with comparison operators, such as =, <, or <>.

We will have to use the IS NULL and IS NOT NULL operators instead.

Tip: Always use IS NULL to look for NULL values.

2. Delete: delete statement is used to delete all records from database table.

There are two types of delete

a) Wild card delete: wild card delete means we can delete complete rows or record from table.

Syntax: delete from tablename;

Example: delete from employee;

If we use this command then all records from employee table get deleted.

```
mysql> select *from employee;
+----+-----+-----+-----+
| id | name  | sal  | email                |
+----+-----+-----+-----+
| 1  | ram   | 10000 | ram@gmail.com        |
| 2  | shyam | 20000 | shyam@gmail.com      |
| 1  | Ganesh | NULL  | NULL                 |
+----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> delete from employee;
Query OK, 3 rows affected (0.03 sec)

mysql> select *from employee;
Empty set (0.03 sec)
mysql>
```

before delete record of employee table

after delete record of employee table.

b) Partial delete: you can delete specific row from database table using some specified conditions.

Syntax: delete from tablename where condition;

Example: we want to delete record of employee whose name is Ganesh so your SQL delete statement look like as

```
mysql> select *from employee;
+----+-----+-----+-----+
| id | name  | sal  | email                |
+----+-----+-----+-----+
| 1  | ram   | 10000 | ram@gmail.com        |
| 2  | shyam | 20000 | shyam@gmail.com      |
| 1  | Ganesh | NULL  | NULL                 |
+----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> delete from employee where name='Ganesh';
Query OK, 1 row affected (0.04 sec)

mysql> select *from employee;
+----+-----+-----+-----+
| id | name  | sal  | email                |
+----+-----+-----+-----+
| 1  | ram   | 10000 | ram@gmail.com        |
| 2  | shyam | 20000 | shyam@gmail.com      |
+----+-----+-----+-----+
2 rows in set (0.00 sec)
```

before delete of employee record

after delete employee record

3. Update: if we want to modify the data of specific column in row then we can use update statement

There are two types of update statement

a) Wild card update: wild card update means we modify the value of all data from specified column

Syntax: update tablename set columnname=value, columnname= value;

Example: we want to update the salary to 50000 of all employees.

```
mysql> select *from employee;
```

id	name	sal	email
1	ram	10000	ram@gmail.com
2	shyam	20000	shyam@gmail.com

```
2 rows in set (0.00 sec)
```

```
mysql> update employee set sal=50000;
```

Query OK, 2 rows affected (0.04 sec)
Rows matched: 2 Changed: 2 Warnings: 0

```
mysql> select *from employee;
```

id	name	sal	email
1	ram	50000	ram@gmail.com
2	shyam	50000	shyam@gmail.com

```
2 rows in set (0.00 sec)
```

b) Partial update: you can update specified column or value using some condition

Syntax: update table name set columnname=value where condition;

Example: we want to update salary 40000 of employee whose id is 1

```
mysql> select *from employee;
```

id	name	sal	email
1	ram	50000	ram@gmail.com
2	shyam	50000	shyam@gmail.com

```
2 rows in set (0.00 sec)
```

```
mysql> update employee set sal=40000 where id=1;
```

Query OK, 1 row affected (0.03 sec)
Rows matched: 1 Changed: 1 Warnings: 0

```
mysql> select *from employee;
```

id	name	sal	email
1	ram	40000	ram@gmail.com
2	shyam	50000	shyam@gmail.com

```
2 rows in set (0.00 sec)
```

4. Select: it is used for fetch data from database table or display data from database table.

There are two types of Select

1) Wild card selection: wild card selection means we can fetch all column data from database table and it is denoted by *.

Syntax: select *from tablename;

Example:

```
mysql> select *from employee;
+----+-----+-----+-----+
| id  | name  | sal  | email                |
+----+-----+-----+-----+
| 1   | ram   | 40000 | ram@gmail.com        |
| 2   | shyam | 50000 | shyam@gmail.com      |
+----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

2) Partial selection: we want to fetch particular column from database table called as partial selection. Suppose consider we have employee table which contains four columns: id, name, sal, and email, and we want to fetch only two columns: id and name, not all columns.

Syntax: select column1, column2.....column...n from tablename;

Example: select id, name from employee;

```
mysql> select id,name from employee;
+----+-----+
| id  | name  |
+----+-----+
| 1   | ram   |
| 2   | shyam |
+----+-----+
2 rows in set (0.00 sec)

mysql> _
```

Constraints in SQL

Q. what is constraints?

Constraints are some rules or restrictions applied on columns of a table or database table called as constraints.

SQL constraints are used to specify rules for the data in a table.

Constraints are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table. If there is any violation between the constraint and the data action, the action is aborted.

Constraints can be column level or table level. Column level constraints apply to a column, and table level constraints apply to the whole table.

The following constraints are commonly used in SQL:

NOT NULL - Ensures that a column cannot have a NULL value

UNIQUE - Ensures that all values in a column are different

PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Uniquely identifies each row in a table

FOREIGN KEY - Prevents actions that would destroy links between tables

CHECK - Ensures that the values in a column satisfies a specific condition

DEFAULT - Sets a default value for a column if no value is specified

CREATE INDEX - Used to create and retrieve data from the database very quickly

There are six types of constraints we have

1. Not null: not null constraints means cannot accept the null value in column called as non null constraints means when user try to insert null value in column then database table should not accept it

Syntax: create table tablename (columnname datatype(size) not null)

Example:

MySQL> create table employee(id int(5), name varchar(200) not null ,email varchar(200) ,contact varchar(20));

```
mysql> create table employee(id int(5), name varchar(200) not null ,email varchar(200) ,contact varchar(20));
Query OK, 0 rows affected, 1 warning (0.06 sec)
```

```
mysql> commit;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> insert into employee (id,email,contact) values(1,'ram@gmail.com','123456789');
ERROR 1364 (HY000): Field 'name' doesn't have a default value
mysql> _
```

If we think about above code we get error field name doesn't have default value.

Because we applied not null constraints on name column so we must have to pass value to name column and when we not applied not null constraints on column then database by default pass null value in it.

```
mysql> select *from employee;
+-----+-----+-----+-----+
| id    | name  | email          | contact |
+-----+-----+-----+-----+
| 1     | ram   | ram@gmail.com  | 123456789 |
| 2     | shyam | ram@gmail.com  | 123456789 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> _
```

If we think about above screen shot we have duplicate email address because but two employee cannot have same email address and if someone try to add duplicate data in column or email column as per our

database then database should not accept duplicate entry for that we need to apply one more constraints on column called as unique

2. Unique: unique key constraints not allow duplicated value in column of database table means must have unique value in database column

The UNIQUE constraint ensures that all values in a column are different.

Both the UNIQUE and PRIMARY KEY constraints provide a guarantee for uniqueness for a column or set of columns.

A PRIMARY KEY constraint automatically has a UNIQUE constraint.

However, you can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.

Syntax to apply unique key constraints

```
create table tablename(columnname datatype(size) unique);
```

or

Note: you can apply more than one constraint on single column

```
create table tablename(columnname datatype(size) not null unique);
```

```
mysql> create table employee(id int(5), name varchar(200) not null ,email varchar(200) unique ,contact varchar(20) not null unique);
Query OK, 0 rows affected, 1 warning (0.12 sec)

mysql> insert into employee values(1,'ram','ram@gmail.com','1234567891');
Query OK, 1 row affected (0.02 sec)

mysql> insert into employee values(1,'ram','ram@gmail.com','1234567891');
ERROR 1062 (23000): Duplicate entry '1234567891' for key 'contact'
mysql> insert into employee values(1,'ram','ram@gmail.com','12345678914');
ERROR 1062 (23000): Duplicate entry 'ram@gmail.com' for key 'email'
mysql>
```

Note: if we think about above screen short we get error for duplicate entry for contact and duplicate entry for email because we maintain unique key with email and contact and we try to insert duplicate data in email and contact column and it is not possible so we get error

3. Primary: primary key is by default not null and unique means when we apply primary key constraints on database table then we cannot pass null value in column as well as cannot pass duplicate value in column called as primary key constraints

The PRIMARY KEY constraint uniquely identifies each record in a table.

Primary keys must contain UNIQUE values, and cannot contain NULL values.

A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).

Syntax: create table tablename (columnname datatype (size) primary key);

```
mysql> create table employee(id int(5) primary key , name varchar(200) not null, email varchar(200) not null unique,contact varchar(200) not null unique);
Query OK, 0 rows affected, 1 warning (0.08 sec)

mysql> insert into employee values(1,'abc','abc@gmail.com','1234567891',
'>');
Query OK, 1 row affected (0.01 sec)

mysql> insert into employee values(1,'abc','abc@gmail.com','1234567891');
ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
mysql>
```

Note: if we think about above screen short we have employee id as primary key so we cannot insert it as duplicate and we try to insert employee id as duplicate so we get error at run time.

Note: you can set primary as auto increment or unique as auto increment means when we not provide the value to primary column then database automatically increase value by 1 and give primary key column value automatically.

How to setup the auto increment with primary key column

Syntax:

create table table name(columnname datatype(size) primary key auto_increment);

Example: create table employee(eid int(5) primary key auto_increment, name varchar(200) not null);

Note: when we setup the auto increment with primary key and then you can use '0' in single quote for primary key column as value then database automatically increase of primary or pass null value primary key column so database automatically increase value of primary key and insert in database column

```
mysql> create table employee(eid int(5) primary key auto_increment, name varchar(200) not null, email varchar(200) not null unique,contact varchar(200) not null unique);
Query OK, 0 rows affected, 1 warning (0.07 sec)

mysql> insert into employee values('0','abc','abc@gmail.com','1234567891');
Query OK, 1 row affected (0.01 sec)

mysql> insert into employee values('0','mno','mno@gmail.com','41234567891');
Query OK, 1 row affected (0.01 sec)

mysql> insert into employee values(null,'pqr','pqr@gmail.com','4134567891');
Query OK, 1 row affected (0.03 sec)

mysql> select *from employee;
+----+-----+-----+-----+
| eid | name | email | contact |
+----+-----+-----+-----+
| 1 | abc | abc@gmail.com | 1234567891 |
| 2 | mno | mno@gmail.com | 41234567891 |
| 3 | pqr | pqr@gmail.com | 4134567891 |
+----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Q. what is diff between primary key and unique column?

Primary key	Unique key
Primary key is by default not null and unique	Unique key is not by default not null
Primary key cannot accept null value	Unique key can accept null value
Single primary key must be present in database table	More than one unique may be present in database table
Primary key use the cluster index or primary	Unique key use the un cluster or secondary index.

4. foreign: foreign key constraints help us to provide the relationship between two different tables means suppose consider if we have two tables and if we want to provide the relationship between them then we can use foreign key constraints

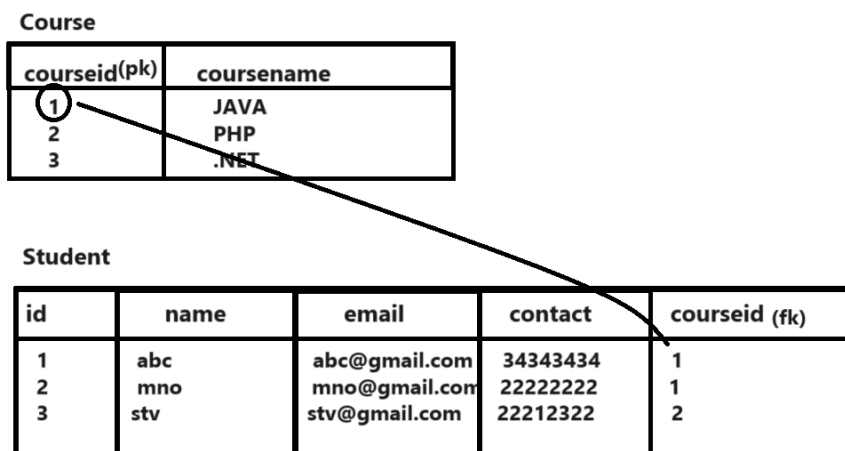
SQL FOREIGN KEY Constraint

The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.

A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table.

The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.

Example: suppose consider we have two table one is course and one is student and course contain courseid and name and course all data present in course table and student contain id, name, email, contact and courseid here we use course id to identify relation between course and student so here course id in student table act as foreign key and courseid in course table act as primary key shown in following diagram.



If we think about above code we have course table which contain courseid and course name and here courseid act as primary key and we use course id in student table but here course id act as foreign key so as per our above table and database we have 1 primary key represent JAVA course and we use 1 primary key in student table in row abc and mno student means we can say abc and mno both student join class means courseid provide relationship between course and student and we use primary value of course table 2 in student stv row so we consider stv is student of PHP.

How to create foreign key practically

create table tablename(columnname datatype(size),foreign key(columnname) references parenttablename(primary key column name));

Note: every foreign key is always primary key in some other place

```
mysql> create table course(cid int(5) primary key auto_increment, name varchar(200) not null unique);
```

Query OK, 0 rows affected, 1 warning (0.10 sec)

```
mysql> create table studnet(sid int(5) primary key auto_increment, name varchar(200) not null, email  
varchar(200) not null unique, contact varchar(200) not null unique,cid int(5),foreign key(cid) references  
course(cid));
```

Query OK, 0 rows affected, 2 warnings (0.14 sec)

```
mysql> insert into course values('0','JAVA');
```

Query OK, 1 row affected (0.01 sec)

```
mysql> insert into course values('0','.NET');
```

Query OK, 1 row affected (0.01 sec)

```
mysql> insert into course values('0','PHP');
```

Query OK, 1 row affected (0.04 sec)

```
mysql> insert into student values('0','abc','abc@gmail.com','123456789',1);
```

ERROR 1146 (42S02): Table 'july2023.student' doesn't exist

```
mysql> insert into studnet values('0','abc','abc@gmail.com','123456789',1);
```

Query OK, 1 row affected (0.04 sec)

```
mysql> insert into studnet values('0','mno','mno@gmail.com','1423456789',1);
```

Query OK, 1 row affected (0.01 sec)

```
mysql> select *from course;
```

```
mysql> select *from course;
+----+-----+
| cid | name  |
+----+-----+
| 2   | .NET  |
| 1   | JAVA  |
| 3   | PHP   |
+----+-----+
3 rows in set (0.00 sec)

mysql> select *from student;
ERROR 1146 (42S02): Table 'july2023.student' doesn't exist
mysql> select *from studnet;
+----+-----+-----+-----+-----+
| sid | name  | email          | contact    | cid |
+----+-----+-----+-----+-----+
| 1   | abc   | abc@gmail.com  | 123456789  | 1   |
| 2   | mno   | mno@gmail.com  | 1423456789 | 1   |
+----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

Note: if we use any primary key as foreign key in table and if we try to delete the primary key value then primary key cannot delete its row or record because its references used in other place means if we want

to delete any primary key record from parent table then we need to delete its child record before parent record shown in following diagram and code.

```
mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 2   | .NET |
| 1   | JAVA |
| 3   | PHP  |
+----+-----+
3 rows in set (0.08 sec)

mysql> select *from studnet;
+----+-----+-----+-----+-----+
| sid | name | email          | contact | cid |
+----+-----+-----+-----+-----+
| 1   | abc  | abc@gmail.com  | 123456789 | 1   |
| 2   | mno  | mno@gmail.com  | 1423456789 | 1   |
+----+-----+-----+-----+-----+
2 rows in set (0.06 sec)

mysql> delete from course where cid=1;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`july2023`.`studnet`, CONSTRAINT `studnet_ibfk_1` FOREIGN KEY (`cid`) REFERENCES `course` (`cid`))
mysql> _
```

If we think about above screen short we try to delete the primary key record from course whose value is 1 means we try to delete the JAVA course from course table and we use the primary key value 1 in studnet table as foreign key means primary key value 1 use in student table so we cannot delete means so if we want to delete the primary key value before that we need to delete the foreign key records from studnet table after that you can delete primary key record shown in following code and diagram.

```
mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 2   | .NET |
| 1   | JAVA |
| 3   | PHP  |
+----+-----+
3 rows in set (0.00 sec)

mysql> select *from studnet;
+----+-----+-----+-----+-----+
| sid | name | email          | contact | cid |
+----+-----+-----+-----+-----+
| 1   | abc  | abc@gmail.com  | 123456789 | 1   |
| 2   | mno  | mno@gmail.com  | 1423456789 | 1   |
+----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> delete from studnet where cid=1;
Query OK, 2 rows affected (0.04 sec)

mysql> delete from course where cid=1;
Query OK, 1 row affected (0.01 sec)

mysql> select *from studnet;
Empty set (0.00 sec)

mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 2   | .NET |
| 3   | PHP  |
+----+-----+
2 rows in set (0.00 sec)
```

if we think about above screen short we delete student record first and after we delete course record means first we delete foreign key record and after we delete primary key record.

Note: it is not possible in real time when we have large number of tables in database and if we use the single primary in multiple tables as foreign key shown in following example.

Course

cid (pk)	name
1	JAVA
2	PHP
3	.NET

Company

compid pk	comp name
1	XYZ
2	PQR

Student

sid (pk)	name	email	contact	cid (fk)
1	abc	abc@	123456	1
2	mno	mno@	444444	1
3	stv	stv@	333333	2

Placement

pid	sid(fk)	package	compid(fk)
1	1	3	1
2	2	4	2

delete from course where cid=1;

If we think about above database if we want to delete the course from course table we before we need to delete from student table related with course id and before student table we need to delete record placement table and before placement we need to delete company if we want to delete course by using this approach you have to write following commands

delete from company where cid=1;

delete from placement where pid=1

delete from student where cid=1

delete from course where cid=1

but it is not good approach and not possible with large database table so avoiding this problem SQL provide three additional constraint to us with foreign key

a) On delete cascade: on delete cascade constraint delete all child record or foreign key record when we delete the primary key record or parent record mean we not need to delete child record manually when we delete primary key record

Syntax: How to use on delete cascade with foreign key

Syntax: create table tablename(columnname datatype(size),foreign key() references parenttablename(columnname) on delete cascade);

```
mysql> create table student(sid int(5) primary key auto_increment,name varchar(200) not null,email
varchar(200) not null unique, contact varchar(200) not null unique,cid int(5),foreign key(cid) references
course(cid) on delete cascade);
```

Query OK, 0 rows affected, 2 warnings (0.10 sec)

```
mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 2   | .NET |
| 1   | JAVA |
| 3   | PHP  |
+----+-----+
3 rows in set (0.00 sec)

mysql> select *from student;
+----+-----+-----+-----+-----+
| sid | name  | email          | contact | cid |
+----+-----+-----+-----+-----+
| 1   | ram   | ram@gmail.com  | 1234567891 | 1 |
| 2   | shyam | shyam@gmail.com | 554444441  | 1 |
| 3   | ganesh | ganesh@gmail.com | 223344555  | 2 |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> delete from course where cid=1;
Query OK, 1 row affected (0.04 sec)

mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 2   | .NET |
| 3   | PHP  |
+----+-----+
2 rows in set (0.00 sec)

mysql> select *from student;
+----+-----+-----+-----+-----+
| sid | name  | email          | contact | cid |
+----+-----+-----+-----+-----+
| 3   | ganesh | ganesh@gmail.com | 223344555 | 2 |
+----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> _
```

if we think about above screen short we have statement delete from course where cid=1 means here we delete the course whose id 1 then automatically child record of primary key 1 get deleted from student table.

b) On update cascade: on update cascade means when we change the value of primary key then foreign key column value should be change in child table automatically called as on update cascade

Syntax: create table tablename(columnname datatype(size),foreign key(columnname) references parenttablename(columnname) on update cascade);

```
mysql> create table student(sid int(5) primary key auto_increment,name varchar(200) not null,email
varchar(200) not null unique, contact varchar(200) not null unique,cid int(5),foreign key(cid) references
course(cid) on update cascade);
```

```
mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 2   | .NET |
| 1   | JAVA |
| 3   | PHP  |
+----+-----+
3 rows in set (0.00 sec)

mysql> select *from student;
+----+-----+-----+-----+-----+
| sid | name  | email                | contact | cid |
+----+-----+-----+-----+-----+
| 1   | ram   | ram@gmail.com        | 1234567891 | 1 |
| 2   | shyam | shyam@gmail.com      | 554444441 | 1 |
| 3   | ganesh | ganesh@gmail.com     | 223344555 | 2 |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

if we think about above screen short we have course name as JAVA with primary key value 1 and we use the value 1 as foreign key in student table so we want to update the primary key value from 1 to 100 means we can update the JAVA course primary value 100 but it should be automatically reflected in student table from 1 to 100 we not update cid of student table manually by code it should be internally update by database engine and it is possible by using on update cascade constraints.

```
mysql> update course set cid=100 where cid=1;
Query OK, 1 row affected (0.05 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 2   | .NET |
| 100 | JAVA |
| 3   | PHP  |
+----+-----+
3 rows in set (0.00 sec)

mysql> select *from student;
+----+-----+-----+-----+-----+
| sid | name  | email                | contact | cid |
+----+-----+-----+-----+-----+
| 1   | ram   | ram@gmail.com        | 1234567891 | 100 |
| 2   | shyam | shyam@gmail.com      | 554444441 | 100 |
| 3   | ganesh | ganesh@gmail.com     | 223344555 | 2 |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

c) On delete set null: on delete set null constraints, when we delete primary key record or parent record then set null value in foreign key record or child record.

create table tablename(columnname datatype(size), foreign key(columnname) references parenttablename(columnname) on delete set null);

Example:

```
mysql> create table student(sid int(5) primary key auto_increment,name varchar(200) not null,email
varchar(200) not null unique, contact varchar(200) not null unique,cid int(5),foreign key(cid) references
course(cid) on delete set null);
```

```
mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 2   | .NET |
| 100 | JAVA |
| 3   | PHP  |
+----+-----+
3 rows in set (0.00 sec)

mysql> select *from student;
+----+-----+-----+-----+-----+
| sid | name  | email          | contact | cid |
+----+-----+-----+-----+-----+
| 1   | Ram   | ram@gmail.com  | 123456789 | 100 |
| 2   | shyam | shyam@gmail.com | 1234546789 | 100 |
| 3   | ganesh | ganesh@gmail.com | 13334546789 | 2   |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

if we think about above diagram we have two tables name as course and student and course table 100 primary key value whose course name is JAVA and we want to delete the course whose id is 100 and we use the on delete set null with cid in student table so when we delete the record with course id 100 then cid in student table must be NULL where 100 value present shown in following diagram.

```
mysql> delete from course where cid=100;
Query OK, 1 row affected (0.01 sec)

mysql> select *from student;
+----+-----+-----+-----+-----+
| sid | name  | email          | contact | cid |
+----+-----+-----+-----+-----+
| 1   | Ram   | ram@gmail.com  | 123456789 | NULL |
| 2   | shyam | shyam@gmail.com | 1234546789 | NULL |
| 3   | ganesh | ganesh@gmail.com | 13334546789 | 2   |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

5. Check: check constraints help us to store the value in column by checking the specified condition.

SQL CHECK Constraint

The CHECK constraint is used to limit the value range that can be placed in a column.

If you define a CHECK constraint on a column it will allow only certain values for this column.

If you define a CHECK constraint on a table it can limit the values in certain columns based on values in other columns in the row.

Syntax: create table tablename (columnname datatype (size) check condition);

Example: suppose consider we have table name as Voter with id name and age column and we want to insert the voter record if age is more than 18

```
mysql> create table voter(vid int(5) primary key auto_increment,name varchar(200) not null, age int(5) check (age>18));
Query OK, 0 rows affected, 2 warnings (0.04 sec)
```

```
mysql> insert into voter values('0','Ram',20);
Query OK, 1 row affected (0.02 sec)

mysql> insert into voter values('0','shyam',10);
ERROR 3819 (HY000): Check constraint 'voter_chk_1' is violated.
mysql>
```

if we think about above insert query we have first statement insert into voter values('0','Ram',20) here we provide value of age is 20 and 20 is greater than 18 means this condition is satisfied and record inserted successfully and we have second insert statement insert into voter values('0','Shyam',10) Here we provide age of voter is 10 and 10 is not greater than 18 so database engine generate the error to us for check constraints

6. default: default constraints means we store default value in column if user not provide some time if we want to set some fix value in column if user not enter data in that column then we can use default constraints and when user provide value to column then user value get override on default value
SQL DEFAULT Constraint

The DEFAULT constraint is used to set a default value for a column.

The default value will be added to all new records, if no other value is specified.

Syntax: create table tablename (columnname datatype(size) default value);

Example: suppose consider we want to create table name as employee with field id, name and salary and salary should be minimum 10000 so means we want to set default value 10000 with column sal.

```
mysql> create table employee(id int(5) primary key auto_increment,name varchar(200),sal int(5) default 10000);
Query OK, 0 rows affected, 2 warnings (0.04 sec)
```

```
mysql> insert into employee (id,name) values(1,'ABC');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select *from employee;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 1  | ABC  | 10000 |
+----+-----+-----+
1 row in set (0.00 sec)
```

If we think about above insert query we insert only id and name not insert salary but we set default value to salary column 10000 so here database engine automatically 10000 value in sal column because it is our default value.

When we user provide value to column then default value not consider user value override on default value shown in following code and diagram.

```
mysql> insert into employee (id,name,sal) values(2,'MNO',50000);
Query OK, 1 row affected (0.01 sec)

mysql> select *from employee;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 1  | ABC  | 10000 |
| 2  | MNO  | 50000 |
+----+-----+-----+
2 rows in set (0.05 sec)
```

If we think about above insert query we provide 50000 value to sal column means 50000 is user value so it is override on default value i.e. on 10000 so we get 50000 in sal column with MNO employee.

Where clause

What is CLAUSE in SQL?

A clause in SQL is a part of a query that lets you filter or customize how you want your data to be queried to you.

The WHERE clause is used to filter records

It is used to extract only those records that fulfil a specified condition.

Where clause is used for apply condition on DML commands

Syntax of where with select command

Select *from tablename where condition;

Syntax of where with delete command

Delete from tablename where condition;

Syntax of where with update command

Update tablename set column=value, column=value where condition;

Example: fetch employee data from employee table whose name is MNO

```
mysql> select *from employee where name='MNO';
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 2  | MNO  | 50000 |
+----+-----+-----+
1 row in set (0.00 sec)
```

Example: Fetch employee record whose salary is greater than 30000

```
mysql> select *from employee where sal>30000;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 2  | MNO  | 50000 |
| 4  | STV  | 40000 |
| 5  | XYZ  | 45000 |
+----+-----+-----+
3 rows in set (0.00 sec)
```

The following operators can be used in the **WHERE** clause:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

Logical Operator in SQL

Operator	Meaning
&&	Logical AND or you can use this operator directly Using and keyword
	Logical OR or you can use this operator directly using or keyword
!	Logical NOT or you can use this operator directly using not keyword

The SQL AND Operator

The WHERE clause can contain one or many AND operators.

The AND operator is used to filter records based on more than one condition, like if you want to return all customers from Spain that starts with the letter 'G':

AND vs OR

The **AND** operator displays a record if *all* the conditions are TRUE.

The **OR** operator displays a record if *any* of the conditions are TRUE.

The SQL OR Operator

The WHERE clause can contain one or more OR operators

The OR operator is used to filter records based on more than one condition, like if you want to return all customers from Germany but also those from Spain:

The NOT Operator

The NOT operator is used in combination with other operators to give the opposite result, also called the negative result.

Example: WAP SQL Query to display employee record whose salary is not 50000

```
mysql> select *from employee where sal!=50000;
```

id	name	sal
1	ABC	10000
3	PQR	30000
4	STV	40000
5	XYZ	45000

4 rows in set (0.00 sec)

```
mysql>
```

```
mysql> select *from employee where not sal=50000;
```

id	name	sal
1	ABC	10000
3	PQR	30000
4	STV	40000
5	XYZ	45000

4 rows in set (0.02 sec)

```
mysql> select *from employee where sal<>50000;
```

id	name	sal
1	ABC	10000
3	PQR	30000
4	STV	40000
5	XYZ	45000

4 rows in set (0.00 sec)

■  negation operator

Example: WAL SQL Query to display the record of employee whose name is ABC and salary is 20000

```
mysql> select *from employee where name='ABC' and sal=10000;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 1  | ABC  | 10000 |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> select *from employee where name='ABC' && sal=10000;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 1  | ABC  | 10000 |
+----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

Example: WA SQL Query to display the employee record whose name is ABC or salary is 10000

```
mysql> select *from employee where name='ABC' or sal=10000;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 1  | ABC  | 10000 |
| 6  | ABC  | 45000 |
+----+-----+-----+
2 rows in set (0.00 sec)

mysql> select *from employee where name='ABC' || sal=10000;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 1  | ABC  | 10000 |
| 6  | ABC  | 45000 |
+----+-----+-----+
2 rows in set, 1 warning (0.00 sec)
```

distinct keyword: distinct keyword is used for fetch unique data from column suppose consider we have to fetch particular column data and column may be contain duplicated data in backend but on output screen we want to show only unique data in this case we can use distinct keyword.

Syntax: select distinct columnname from tablename;

Example: we want to fetch only name of employees from employee table.

```
mysql> select name from employee;
+-----+
| name |
+-----+
| ABC  |
| MNO  |
| PQR  |
| STV  |
| XYZ  |
| ABC  |
+-----+
6 rows in set (0.00 sec)
```

If we think about above output we have ABC display two times on output screen means ABC is duplicated on output screen but if we want to show the ABC single time on output screen then we can use distinct keyword

```
mysql> select distinct name from employee;
+-----+
| name |
+-----+
| ABC  |
| MNO  |
| PQR  |
| STV  |
| XYZ  |
+-----+
5 rows in set (0.02 sec)
```

Note : to copy all table data into another table

create table abc as select * from employee;

IN and between operator

IN operator is use for avoid to write the multiple OR conditions means some time if we required to write the multiple or condition using single column then writing multiple OR is not good approach so you can use IN operator other than writing multiple OR conditions.

The IN operator allows you to specify multiple values in a WHERE clause.

The IN operator is a shorthand for multiple OR conditions.

Syntax: select *from tablename where columnname IN (value1, value2.....value....n);

Example: WA SQL Query to display the record of employee whose salary 10000, 30000, 45000 as well as 20000

```
mysql> select *from employee where sal=10000 or sal=30000 or sal=45000 or sal=20000;
```

id	name	sal
1	ABC	10000
3	PQR	30000
5	XYZ	45000
6	ABC	45000

4 rows in set (0.00 sec)

If we think about above statement we use three OR condition in SQL select statement

So writing multiple or is not good we have one alternative or short cut for avoid multiple OR condition called as IN operator.

```
mysql> select *from employee where sal IN(10000,30000,45000,20000);
```

id	name	sal
1	ABC	10000
3	PQR	30000
5	XYZ	45000
6	ABC	45000

4 rows in set (0.00 sec)

Note: IN Operator work as conjunction in sub query or inner query.

Example WA SQL Query to display the employee record whose name is ABC, PQR, and XYZ

```
mysql> select *from employee where name IN('ABC','PQR','XYZ');
```

id	name	sal
1	ABC	10000
3	PQR	30000
5	XYZ	45000
6	ABC	45000

4 rows in set (0.00 sec)

Between Operator

Between operators is used for fetch data between two specified range values.

The BETWEEN operator selects values within a given range the values can be numbers, text, or dates.

The BETWEEN operator is inclusive: begin and end values are included.

Syntax: select *from tablename where columnname between value1 and value2

Example: WA SQL query to display employee record whose salary between 30000 to 50000

```
mysql> select *from employee where sal between 30000 and 50000;
```

id	name	sal
2	MNO	50000
3	PQR	30000
4	STV	40000
5	XYZ	45000
6	ABC	45000

5 rows in set (0.00 sec)

If we think about above query it is similar as given screen short

```
mysql> select *from employee where sal>=30000 and sal<=50000;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 2  | MNO  | 50000 |
| 3  | PQR  | 30000 |
| 4  | STV  | 40000 |
| 5  | XYZ  | 45000 |
| 6  | ABC  | 45000 |
+----+-----+-----+
5 rows in set (0.00 sec)
```

Aggregate function of SQL

Aggregate function help us to compare all column values and generate combine single result it may be result with all column value or it may be single value by comparison with multiple column values.

There are five types of Aggregate Function

count(): count function is used for count the total number of record present in database table.

There are two ways to use count function

a) Skip the null values: means this type of count not considers value counting whose value is null.

Syntax: select count(columnname) from tablename;

Example: WA SQL Query to count the number of employee from employee table.

```
mysql> select *from employee;
+----+-----+-----+
| id | name | sal  |
+----+-----+-----+
| 1  | ABC  | 10000 |
| 2  | MNO  | 50000 |
| 3  | PQR  | 30000 |
| 4  | STV  | 40000 |
| 5  | XYZ  | 45000 |
| 6  | ABC  | 45000 |
| 7  | NULL | 8000  |
| 8  | NULL | 9000  |
+----+-----+-----+
8 rows in set (0.00 sec)

mysql> select count(name) from employee;
+-----+
| count(name) |
+-----+
| 6           |
+-----+
1 row in set (0.03 sec)
```

If we think about the above SQL query we get result count with 6 but we have 8 records in employee table because we have two names with null value and when we pass column name in count then not count the null values.

b) Count null value as well as non null value: when we want to count null values as well as not null values we can use * in count function.

Syntax: select count (*) from tablename;

Example:

```
mysql> select count(*) from employee;
+-----+
| count(*) |
+-----+
|         8 |
+-----+
1 row in set (0.00 sec)
```

sum() : sum() function is used for calculate the sum of column values

Syntax: select sum(columnname) from tablename;

Example: count the sum of salary of all employees.

```
mysql> select sum(sal) from employee;
+-----+
| sum(sal) |
+-----+
|    237000 |
+-----+
1 row in set (0.00 sec)

mysql> select *from employee;
+----+-----+-----+
| id | name  | sal  |
+----+-----+-----+
|  1 | ABC   | 10000 |
|  2 | MNO   | 50000 |
|  3 | PQR   | 30000 |
|  4 | STV   | 40000 |
|  5 | XYZ   | 45000 |
|  6 | ABC   | 45000 |
|  7 | NULL  | 8000  |
|  8 | NULL  | 9000  |
+----+-----+-----+
8 rows in set (0.00 sec)
```

avg() : this function help us to calculate the average value of column (not consider null value)

Syntax: select avg(columnname) from tablename;

Example: Calculate the average salary of employee;

Example: select avg(sal) from employee;

```
mysql> select sum(sal)/8 from employee;
+-----+
| sum(sal)/8 |
+-----+
| 29625.0000 |
+-----+
1 row in set (0.00 sec)

mysql> select sum(sal)/count(name) from employee;
+-----+
| sum(sal)/count(name) |
+-----+
|          39500.0000 |
+-----+
1 row in set (0.00 sec)
```

min() : this function is used for return the min value from database column.

The MIN() function returns the smallest value of the selected column.

Syntax: select min(columnname) from tablename;

Example: find the minimum salary of employee;

```
mysql> select min(sal) from employee;
+-----+
| min(sal) |
+-----+
|      8000 |
+-----+
1 row in set (0.04 sec)
```

The MAX() function returns the largest value of the selected column

Syntax:

select max(columnname) from tablename;

Example: select max(sal) from employee;

```
mysql> select max(sal) from employee;
+-----+
| max(sal) |
+-----+
|     50000 |
+-----+
1 row in set (0.00 sec)
```

group by clause and having clause

Group by clause: this clause helps us to group the similar value by using particular column
Like as count the dept wise employee, count the city wise voter list etc.

The GROUP BY statement is often used with aggregate functions (COUNT (), MAX (), MIN (), SUM (), AVG ()) to group the result-set by one or more columns.

Syntax of group by is: select columnname from tablename group by columnname;

If we want to work with group by we have some important points.

1. Group by use with column
2. You can use column name in select command which we use in group by
3. Normally aggregate function use with group by clause

Example: Suppose consider we have employee table we want to count the employee list who is taking similar salary.

```
mysql> select sal,count(*) from employee group by sal;
```

sal	count(*)
10000	2
50000	1
30000	2
40000	1
45000	2
8000	1
9000	1
NULL	1

8 rows in set (0.00 sec)

Example: WA SQL create table name as customer with field id, name, city and age and count the Customer who having same age

```
mysql> select *from customer;
```

cid	name	city	age
1	a	pune	20
2	b	nashik	21
3	c	pune	20
4	d	pune	21
5	e	pune	2
6	f	pune	20

6 rows in set (0.00 sec)

```
mysql> select age,count(age) from customer group by age;
```

age	count(age)
20	3
21	2
2	1

3 rows in set (0.00 sec)

Example: WA SQL Query to count customer city wise.

```
mysql> select *from customer;
```

cid	name	city	age
1	a	pune	20
2	b	nashik	21
3	c	pune	20
4	d	pune	21
5	e	pune	2
6	f	pune	20

6 rows in set (0.00 sec)

```
mysql> select city,count(city) from customer group by city;
```

city	count(city)
pune	5
nashik	1

2 rows in set (0.00 sec)

Having clause: having clause is used for check the condition using aggregate function means if we use the where clause and if you have scenario where we required to use aggregate function to use as condition then we cannot use it using where clause for that you can use having clause.

The HAVING clause was added to SQL because the WHERE keyword cannot be used with aggregate functions.

Syntax: select *from tablename where condition group by columnname having condition;

Example: WA SQL query to display salary in table whose is occur more than once

```
mysql> select sal,count(sal) from employee group by sal;
```

sal	count(sal)
10000	2
50000	1
30000	2
40000	1
45000	2
8000	1
9000	1
NULL	0

```
8 rows in set (0.00 sec)

mysql> select sal,count(sal) from employee where count(sal)>1 group by sal;
ERROR 1111 (HY000): Invalid use of group function
mysql> -
```

if we think about above query we get error invalid use of group functions because we use the count function or aggregate function with where clause and it is not possible so here you can use having clause for check the condition.

```
mysql> select sal,count(sal) from employee group by sal having count(sal)>1;
```

sal	count(sal)
10000	2
30000	2
45000	2

```
3 rows in set (0.00 sec)
```

Rules of having clause

1. Having must be used with group by clause
2. Having can work with aggregate function as condition
3. Having clause can use the column name in condition which column use in group or in select query or with aggregate function.
4. Having clause must be writing after group by and before order by clause

Example: WAP SQL query count the customer age wise and whose age occurrence more than one and age should be greater 20

```
mysql> select *from customer;
```

cid	name	city	age
1	a	pune	20
2	b	nashik	21
3	c	pune	20
4	d	pune	21
5	e	pune	2
6	f	pune	20
7	g	nashil	3
8	g	nashil	2

```
8 rows in set (0.00 sec)

mysql> select age from customer where age>20 group by age having count(age)>1;
```

age
21

```
1 row in set (0.01 sec)

-
```

Example: WA SQL Query to find customer whose is 20,21,3

```
mysql> select *from customer where age IN(20,21,3);
```

cid	name	city	age
1	a	pune	20
2	b	nashik	21
3	c	pune	20
4	d	pune	21
6	f	pune	20
7	g	nashik	3

6 rows in set (0.01 sec)

order by clause: The ORDER BY keyword is used to sort the result-set in ascending or descending order.

Syntax: select *from tablename order by column name asc | desc;

Note: when we have to use all clause in single query means where, group by, having and order then your sequence is like as

Select *from tablename where condition group by column name having condition order by column name asc | desc

Example: WA SQL query to display the employee details in descending order by salary

```
mysql> select *from employee order by sal desc;
```

id	name	sal
2	MNO	50000
5	XYZ	45000
6	ABC	45000
4	STV	40000
3	PQR	30000
11	TUXA	30000
1	ABC	10000
10	raghav	10000
8	NULL	9000
7	NULL	8000
9	xyz	NULL

11 rows in set (0.00 sec)

Example: WA SQL query to count the similar salary from table and whose occurrence is more than once and salary should be greater than 9000 and display all salaries in descending order.

```
mysql> select sal ,count(sal) from employee where sal>9000 group by sal having count(sal)>1 order by sal desc;
```

sal	count(sal)
45000	2
30000	2
10000	2

3 rows in set (0.00 sec)

Difference Between ORDER BY and GROUP BY Clause

GROUP BY	ORDER BY
Group by statement is used to group the rows that have the same value.	Whereas Order by statement sort the result-set either in ascending or descending order.
It may be allowed in CREATE VIEW statement.	While it does not use in CREATE VIEW statement.
In select statements, it is always used before the order by keyword.	While in the select statement, it is always used after the group by keyword.
An attribute cannot be in the group by a statement under the aggregate function.	Whereas in order by statement, the attribute can be under aggregate function.
In group by clause, the tuples are grouped based on the similarity between the attribute values of tuples.	Whereas in order by clause, the result set is sorted based on ascending or descending order.
Group by controls the presentation of tuples(rows) .	While order by clause controls the presentation of columns.

Difference between Where and Having Clause in SQL :

SR.NO.	WHERE Clause	HAVING Clause
1.	WHERE Clause is used to filter the records from the table based on the specified condition.	HAVING Clause is used to filter record from the groups based on the specified condition.
2.	WHERE Clause can be used without GROUP BY Clause	HAVING Clause cannot be used without GROUP BY Clause
3.	WHERE Clause implements in row operations	HAVING Clause implements in column operation
4.	WHERE Clause cannot contain aggregate function	HAVING Clause can contain aggregate function
5.	WHERE Clause can be used with SELECT, UPDATE, DELETE statement.	HAVING Clause can only be used with SELECT statement.
6.	WHERE Clause is used before GROUP BY Clause	HAVING Clause is used after GROUP BY Clause
7.	WHERE Clause is used with single row function like UPPER, LOWER etc.	HAVING Clause is used with multiple row function like SUM, COUNT etc.

Q. Why where clause cannot used with aggregate functions

We cannot use the WHERE clause with aggregate functions because it works for filtering individual rows. In contrast, HAVING can work with aggregate functions because it is used to filter groups.

Like operator

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

like operator is used for perform pattern matching in select query means some time if we want to data from database table using some specified pattern then you can use like operator

Means suppose consider we want fetch employee data whose name start with r and ends with sh or find the employee whose name contain three letter etc.

This type of query you can execute by using like operator.

Syntax: select *from tablename where columnname like pattern;

There are two wildcards often used in conjunction with the LIKE operator:

If we want to work like operator we have two wild card characters for match pattern.

% : this character helps us to match one or more than one character in pattern

_ : this character helps us to match single character in pattern.

Example: WA SQL Query to fetch employee details whose name start with r

```
mysql> select *from employee;
```

id	name	sal
1	ABC	10000
2	MNO	50000
3	PQR	30000
4	STV	40000
5	XYZ	45000
6	ABC	45000
7	NULL	8000
8	NULL	9000
9	xyz	NULL
10	raghav	10000
11	TUXA	30000

11 rows in set (0.09 sec)

```
mysql> select *from employee where name like 'r%';
```

id	name	sal
10	raghav	10000

1 row in set (0.03 sec)

it is your original table

Note: if we think about left hand side query the query r% means starting letter should be r after that pattern accept any letter or character

like as
ram
Ramesh
Raghav
etc

Query result

Q. WA SQL Query to fetch employee whose name ends with tv

```
mysql> select *from employee where name like '%tv';
```

id	name	sal
4	STV	40000

1 row in set (0.00 sec)

Q. WA SQL Query to fetch employee whose name contain gh any where

```
mysql> select *from employee where name like '%gh%';
```

id	name	sal
10	raghav	10000

1 row in set (0.00 sec)

Q. WA SQL Query to fetch employee whose name contain more than three letters

Here we required use **_** (underscore character) for identify single letter so your query like as

select *from employee where name like '____%';

Here we use three underscore which represents three character and % represent more than one characters

```
mysql> select *from employee;
```

id	name	sal
1	ABC	10000
2	MNO	50000
3	PQR	30000
4	STV	40000
5	XYZ	45000
6	ABC	45000
7	NULL	8000
8	NULL	9000
9	xyz	NULL
10	raghav	10000
11	TUXA	30000
12	a	10000

12 rows in set (0.00 sec)

```
mysql> select *from employee where name like '____%';
```

id	name	sal
1	ABC	10000
2	MNO	50000
3	PQR	30000
4	STV	40000
5	XYZ	45000
6	ABC	45000
9	xyz	NULL
10	raghav	10000
11	TUXA	30000

Example: WA SQL Query to fetch employee whose name contain single character

```
mysql> select *from employee where name like '_';
```

id	name	sal
12	a	10000

1 row in set (0.00 sec)

Aliases

SQL aliases are used to give a table or a column in a table, a temporary name.

Aliases are often used to make column names more readable.

An alias only exists for the duration of that query

An alias is created with the AS keyword.

JOIN

A JOIN clause is used to combine rows from two or more tables, based on a related column between them.

Q. what is joins and why use it?

Join is used for fetch data from different tables those are interconnected with each other.

Means internally we have data present in different tables or stored in different tables in database but on output screen we want to fetch data and show as single table then we can use joins

Example: Suppose consider we have two tables name as Course and Student and we want to show the output like as

Course			Output		
cid (pk)	name	fees	Student name	coursename	fees
1	JAVA	40000	abc	JAVA	40000
2	PHP	30000	mno	JAVA	40000
3	.NET	40000	pqr	PHP	30000
			stv	JAVA	40000

Student				
sid	name	email	contact	cid (fk)
1	abc	abc@gmail.com	123456789	1
2	mno	mno@gmail.com	333333333	1
3	pqr	pqr@gmail.com	2323245555	2
4	stv	stv@gmail.com	11111111	1

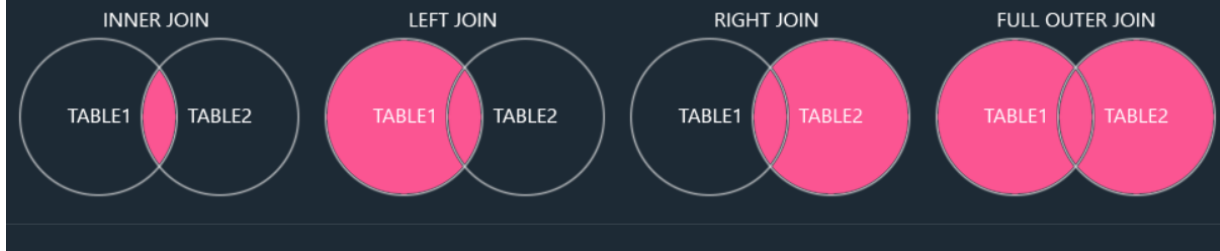
if we think about above diagram we have two tables one is course and one is student and course contain course information and student contain student information and course and student contain one common column i.e. cid and if we think output we want to show student and course detail on single output screen but they are stored separately in different table and for this purpose we can write join queries

Types of Join

Different Types of SQL JOINS

Here are the different types of the JOINS in SQL:

- **(INNER) JOIN** : Returns records that have matching values in both tables
- **LEFT (OUTER) JOIN** : Returns all records from the left table, and the matched records from the right table
- **RIGHT (OUTER) JOIN** : Returns all records from the right table, and the matched records from the left table
- **FULL (OUTER) JOIN** : Returns all records when there is a match in either left or right table



1. **Inner Join**: inner join means fetch common data from left hand side and right hand side table
Returns records that have matching values in both tables

Course

cid (pk)	name	fees
1	JAVA	40000
2	PHP	30000
3	.NET	40000

Output

Student name	coursename	fees
abc	JAVA	40000
mno	JAVA	40000
pqr	PHP	30000
stv	JAVA	40000

Student

sid	name	email	contact	cid (fk)
1	abc	abc@gmail.com	123456789	1
2	mno	mno@gmail.com	333333333	1
3	pqr	pqr@gmail.com	2323245555	2
4	stv	stv@gmail.com	11111111	1

2. **Left Join**: left join means fetch all data from left hand side table and common data from right hand side table.

Returns all records from the left table, and the matched records from the right table

Course

cid (pk)	name	fees
1	JAVA	40000
2	PHP	30000
3	.NET	40000

Output

Student name	coursename	fees
abc	JAVA	40000
mno	JAVA	40000
pqr	PHP	30000
stv	JAVA	40000
NULL	.NET	40000

Student

sid	name	email	contact	cid (fk)
1	abc	abc@gmail.com	123456789	1
2	mno	mno@gmail.com	333333333	1
3	pqr	pqr@gmail.com	2323245555	2
4	stv	stv@gmail.com	11111111	1

3. **Right Join:** right join means fetch all data from right hand side table and common data from left hand side table.

Returns all records from the right table, and the matched records from the left table

Course

cid (pk)	name	fees
1	JAVA	40000
2	PHP	30000
3	.NET	40000

Output

Student name	coursename	fees
abc	JAVA	40000
mno	JAVA	40000
pqr	PHP	30000
stv	JAVA	40000
xyz	NULL	NULL

Student

sid	name	email	contact	cid (fk)
1	abc	abc@gmail.com	123456789	1
2	mno	mno@gmail.com	333333333	1
3	pqr	pqr@gmail.com	2323245555	2
4	stv	stv@gmail.com	11111111	1
5	xyz	xyz@gmail.com	33333333	NULL

4. **Outer Join:** outer join means fetch all data from left hand side and all data from right hand side table means we can say outer join combination of left and right join

FULL (OUTER) JOIN: Returns all records when there is a match in either left or right table

Course

cid (pk)	name	fees
1	JAVA	40000
2	PHP	30000
3	.NET	40000

Output

Student name	coursename	fees
abc	JAVA	40000
mno	JAVA	40000
pqr	PHP	30000
stv	JAVA	40000
xyz	NULL	NULL
NULL	.NET	40000

Student

sid	name	email	contact	cid (fk)
1	abc	abc@gmail.com	123456789	1
2	mno	mno@gmail.com	333333333	1
3	pqr	pqr@gmail.com	2323245555	2
4	stv	stv@gmail.com	11111111	1
5	xyz	xyz@gmail.com	33333333	NULL

5. **Self join:** when we perform join with same table called as self join.

How to perform join practically

When we want to perform join with two tables then table must have common column means left hand side and right side table must have common column for fetch data.

Syntax of join

Select ltref.fieldname, rtref.fieldname from left tablename ltref join type right tablename rtref on ltref.columnname=rtref.columnname;

Consider we have following tables and we want all types of joins on this table.

```
mysql> select *from course;
+----+-----+-----+
| cid | name  | fees  |
+----+-----+-----+
| 1   | JAVA  | 40000 |
| 2   | PHP   | 40000 |
| 3   | .NET  | 40000 |
+----+-----+-----+
3 rows in set (0.00 sec)

mysql> select *from student;
+----+-----+-----+-----+-----+
| sid | name  | email          | contact | cid |
+----+-----+-----+-----+-----+
| 1   | abc   | abc@gmail.com  | 123456789 | 1   |
| 2   | mno   | mno@gmail.com  | 123456739 | 1   |
| 3   | pqr   | pqr@gmail.com  | 1234567339 | 2   |
| 4   | stv   | stv@gmail.com  | 1234567339 | NULL |
+----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Example of inner join;

```
mysql> select *from course;
+----+-----+-----+
| cid | name  | fees  |
+----+-----+-----+
| 1   | JAVA  | 40000 |
| 2   | PHP   | 40000 |
| 3   | .NET  | 40000 |
+----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> select *from student;
+----+-----+-----+-----+-----+
| sid | name  | email          | contact | cid |
+----+-----+-----+-----+-----+
| 1   | abc   | abc@gmail.com  | 123456789 | 1   |
| 2   | mno   | mno@gmail.com  | 123456739 | 1   |
| 3   | pqr   | pqr@gmail.com  | 1234567339 | 2   |
| 4   | stv   | stv@gmail.com  | 1234567339 | NULL |
+----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> select s.name,c.name,c.fees from course c inner join student s on c.cid=s.cid;
+-----+-----+-----+
| name  | name  | fees  |
+-----+-----+-----+
| abc   | JAVA  | 40000 |
| mno   | JAVA  | 40000 |
| pqr   | PHP   | 40000 |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

Example of left join

```
mysql> select s.name,c.name,c.fees from course c left join student s on c.cid=s.cid;
+-----+-----+-----+
| name  | name  | fees  |
+-----+-----+-----+
| abc   | JAVA  | 40000 |
| mno   | JAVA  | 40000 |
| pqr   | PHP   | 40000 |
| NULL  | .NET  | 40000 |
+-----+-----+-----+
4 rows in set (0.04 sec)
```

Example of right join

```
mysql> select s.name,c.name,c.fees from course c right join student s on c.cid=s.cid;
+-----+-----+-----+
| name  | name  | fees  |
+-----+-----+-----+
| abc   | JAVA  | 40000 |
| mno   | JAVA  | 40000 |
| pqr   | PHP   | 40000 |
| stv   | NULL  | NULL  |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Example of outer join

outer join means it is combination of left and right join but if we think about MYSQL there is no outer join keyword for execute outer join so if we want to execute outer join in MySQL we have to write left join and right join separately and combine them using union operator means when we execute the left join and right join using union operator then there is outer join shown in following code.

```
mysql> select s.name,c.name,c.fees from course c left join student s on c.cid=s.cid union select s.name,c.name,c.fees from course c right join student s on c.cid=s.cid;
```

name	name	fees
abc	JAVA	40000
mno	JAVA	40000
pqr	PHP	40000
NULL	.NET	40000
stv	NULL	NULL

5 rows in set (0.04 sec)

```
mysql>
```

Select s.name, c.name, c.fees from course c left join student s on c.cid=s.cid union select s.name, c.name, c.fees from course c right join student s on c.cid=s.cid;

Example: WA SQL Query to display the course name who having more than one admission

```
mysql> select c.name, count(s.cid) from course c inner join student s on c.cid=s.cid group by c.name having count(s.cid)>1;
```

name	count(s.cid)
JAVA	2

1 row in set (0.04 sec)

```
mysql>
```

Select c.name, count(s.cid) from course c inner join student s on c.cid=s.cid group by c.name having count(s.cid)>1;

```
mysql> select c.name as 'Course Name', count(s.cid) as 'Number of Admission' from course c inner join student s on c.cid=s.cid group by c.name having count(s.cid)>1;
```

Course Name	Number of Admission
JAVA	2

1 row in set (0.00 sec)

```
mysql>
```

The SQL UNION Operator

The UNION operator is used to combine the result-set of two or more SELECT statements.

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;
```

Every SELECT statement within UNION must have the same number of columns

The columns must also have similar data types

The columns in every SELECT statement must also be in the same order

UNION ALL Syntax

The UNION operator selects only distinct values by default. To allow duplicate values, use UNION ALL:

```
SELECT column_name(s) FROM table1
UNION ALL
SELECT column_name(s) FROM table2;
```

Note: The column names in the result-set are usually equal to the column names in the first SELECT statement.

Practice Query

Create the database name as empdb with a two tables department and employee

Table Description

Department

Column Name	Data Type	Constraints
Did	int	primary key with auto increment
Dname	varchar	not null and unique

Employee Table

Column Name	data type	constraints
Eid	int	primary key with auto_increment
Name	varchar	not null
Email	varchar	not null and unique
Contact	varchar	not null and unique
Salary	int	not null default 10000
Did	int	foreign key on delete cascade and on

Tables

```
mysql> create table dept(did int(5) primary key auto_increment,name varchar(200) not null unique);
```

```
Query OK, 0 rows affected, 1 warning (0.13 sec)
```

```
mysql> create table employee(eid int(5) primary key auto_increment,name varchar(200) not null, email
varchar(200) not null unique, contact varchar(200) not null unique, salary int(5) default 10000, did
int(5),foreign key(did) references dept(did) on delete cascade on update cascade);
```

Query OK, 0 rows affected, 3 warnings (0.11 sec)

Original Table Structure

```
mysql> select *from dept;
```

did	name
1	DEV
2	PROD
3	TEST ENV

3 rows in set (0.00 sec)

```
mysql> select *from employee;
```

eid	name	email	contact	salary	did
1	Ram	ram@gmail.com	123456789	10000	1
2	shyam	shyam@gmail.com	1234563789	20000	1
3	ganesh	ganesh@gmail.com	12345363789	10000	1
4	rajesh	rajesh@gmail.com	12345363229	20000	2
5	dinesh	dinesh@gmail.com	72345363229	40000	2

5 rows in set (0.00 sec)

Q1. WA SQL Query to perform inner join between employee and department table.

```
mysql> select d.name ,e.name,e.email,e.contact,e.salary from dept d inner join employee e on e.did=d.did;
```

name	name	email	contact	salary
DEV	Ram	ram@gmail.com	123456789	10000
DEV	shyam	shyam@gmail.com	1234563789	20000
DEV	ganesh	ganesh@gmail.com	12345363789	10000
PROD	rajesh	rajesh@gmail.com	12345363229	20000
PROD	dinesh	dinesh@gmail.com	72345363229	40000

5 rows in set (0.00 sec)

Select d.name,e.name,e.email,e.contact,e.salary from dept d inner join employee e on d.did=e.did;

Q2. WA SQL Query to show the employee details with department name whose department is DEV.

```
mysql> select d.name ,e.name,e.email,e.contact,e.salary from dept d inner join employee e on e.did=d.did where d.name='DEV';
```

name	name	email	contact	salary
DEV	Ram	ram@gmail.com	123456789	10000
DEV	shyam	shyam@gmail.com	1234563789	20000
DEV	ganesh	ganesh@gmail.com	12345363789	10000

3 rows in set (0.01 sec)

Select d.name, e.name, e.email, e.contact, e.salary from dept d inner join employee e on d.did=e.did where d.name='dev';

Q3. WA SQL Query to show the employee details with department name whose department is DEV and salary greater than 10000

```
mysql> select d.name ,e.name,e.email,e.contact,e.salary from dept d inner join employee e on e.did=d.did where d.name='DEV' and e.salary>10000;
```

name	name	email	contact	salary
DEV	shyam	shyam@gmail.com	1234563789	20000

1 row in set (0.00 sec)

Q4. WA SQL Query to show the employee details with department details whose department is DEV and name end with sh

```
mysql> select d.name ,e.name,e.email,e.contact,e.salary from dept d inner join employee e on e.did=d.did where d.name='DEV' and e.name like '%sh';
```

name	name	email	contact	salary
DEV	ganesh	ganesh@gmail.com	12345363789	10000

1 row in set (0.00 sec)

Q5. WA SQL Query to display the dept list who having more than 2 employees.

Select d.name, count(e.did) from dept d inner join employee e on e.did=d.did group by d.name having count(e.did)>2

```
mysql> select d.name as 'Dept Name',count(e.did) as 'Employee Count' from dept d left join employee e on e.did=d.did group by d.name having count(e.did)>2;
```

Dept Name	Employee Count
DEV	3

1 row in set (0.04 sec)

Q6. WA SQL Query to display the employee count dept wise whose salary is greater than 10000 and count is less than 3

```
mysql> select d.name,count(e.did) from dept d left join employee e on d.did=e.did where e.salary>10000 group by d.name having count(e.did)<2 ;
```

name	count(e.did)
DEV	1

1 row in set (0.00 sec)

How to write joins with three tables

When we want to use join with three table then we required one intermediate which contain common column of left and right table

```
mysql> create table course(cid int(5) primary key auto_increment,name varchar(200) not null unique);  
Query OK, 0 rows affected, 1 warning (0.07 sec)
```

```
mysql> create table student(sid int(5) primary key auto_increment,name varchar(200) not null,email  
varchar(200) not null unique, contact varchar(200) not null unique);
```

```
mysql> create table  
coursestudjoin(cid int(5),foreign key(cid) references course(cid), sid int(5),foreign key(sid) references  
student(sid));  
Query OK, 0 rows affected, 2 warnings (0.12 sec)
```

```

mysql> select *from course;
+----+-----+
| cid | name |
+----+-----+
| 3   | .NET |
| 1   | JAVA |
| 2   | PHP  |
+----+-----+
3 rows in set (0.00 sec)

mysql> select *from student;
+----+-----+-----+-----+
| sid | name | email | contact |
+----+-----+-----+-----+
| 1   | ram  | ram@gmail.com | 123456789 |
| 2   | shyam | shyam@gmail.com | 723456789 |
| 3   | ganesh | ganesh@gmail.com | 823456789 |
+----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select *from coursestudjoin;
+----+-----+
| cid | sid |
+----+-----+
| 1   | 1   |
| 1   | 2   |
| 2   | 3   |
+----+-----+
3 rows in set (0.00 sec)

```

```
mysql> select c.name as 'Course Name',s.name as 'Student Name' from course c
inner join coursestudjoin csj on c.cid=csj.cid inner join student s on s.sid=csj.sid;
```

```

+-----+-----+
| Course Name | Student Name |
+-----+-----+
| JAVA        | ram          |
| JAVA        | shyam       |
| PHP         | ganesh      |
+-----+-----+
3 rows in set (0.00 sec)

```

Example: WA SQL to show the student wise course whose name start with r

```
mysql> select s.name as 'Student name' from course c inner join coursestudjoin csj on c.cid=csj.cid inner
join student s on s.sid=csj.sid where s.name like 'r%' group by c.name;
select s.name, c.name from student s inner join coursestudjoin csj on s.sid=csj.sid inner join course c
on c.cid=csj.cid where s.name like 'r%' group by c.name;
select s.name,c.name from student s inner join coursestudjoin csj on csj.sid=s.sid inner join course c
on c.cid=csj.cid where s.name like "r%" group by c.name;
```

Normalization

Normalization is technique where we can decompose the large database table in to smaller tables for avoid data duplication or avoid data redundancy

What is Normalization?

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate undesirable characteristics like Insertion, Update, and Deletion Anomalies.
- Normalization divides the larger table into smaller and links them using relationships.
- The normal form is used to reduce redundancy from the database table.

Why do we need Normalization?

The main reason for normalizing the relations is removing these anomalies. Failure to eliminate anomalies leads to data redundancy and can cause data integrity and other

problems as the database grows. Normalization consists of a series of guidelines that helps to guide you in creating a good database structure.

Q. What happen if not normalize the database or what happen without normalization with tables?

If we not normalize database then we have some anomalies or confusion about data insertion, deletion and updation of data

Example:

admission

rollno	name	branch	hod	office_tel
101	abc	CSE	Mr.X	1234567
102	MNO	CSE	Mr.X	1234567
103	PQR	MECH	Mr.Y	3434343
104	STV	MECH	Mr.Y	3434343
105	XYZ	NULL	NULL	NULL

If we think about above table we have insertion anomalies

Insertion anomalies: Insertion Anomaly refers to when one cannot insert a new tuple into a relationship due to lack of data.

Suppose we have student XYZ want to take admission but confuse about branch selection so there is two options if we insert data XYZ then we need to set branch null, if branch is so HOD also null and tel also null otherwise not insert data of student in admission

Deletion anomalies: The delete anomaly refers to the situation where the deletion of data results in the unintended loss of some other important data.

if we delete the record by using HOD then branch record and student record get deleted so it not possible real time means dept is not dependent on HOD if HOD leave the dept then management can replace HOD but student data should be present in database it should not dependent on HOD data.

Updation anomalies: The update anomaly is when an update of a single data value requires multiple rows of data to be updated.

If we think about above table we have updation anomalies suppose consider Mr.x is HOD of CSE and if Mr.x leaves the JOB and management hire the new HOD Mr.z as replacement of Mr.X and suppose admission table contain 240 students of CSE dept so we required to execute update query 240 times with every students.

If we want to solve this problem you can perform normalization with database table

Normalization works through a series of stages called Normal forms. The normal forms apply to individual relations. The relation is said to be in particular normal form if it satisfies constraints.

Types of normalization

1. 1NF
2. 2NF
3. 3NF
4. 4NF
5. Boyce code
- Etc.

First Normal Form

If we want to work with first normal form we have following rules.

1. Table should have single (atomic) value in attribute / column
2. Value stored in column should be of same domain
3. All columns in table should have unique names
4. The order in which data is stored should not matter.

Example: suppose consider we have following examples.

emp_id	emp_name	emp_mobile	emp_skill
1	ABC	88776655	c,c++,java
2	MNO	77889966	java,python
3.	PQR	44556677	.NET

if we think about above table it is not present in first normal form because this table not satisfy the first rule because if we think emp_skill column then we insert multiple data in column by separating comma but rule say column must have single value so this rule not satisfy by above table

If we want to arrange this table in first normal form it looks like as

emp_id	emp_name	emp_mobile	emp_skill
1	ABC	88776655	c
1	ABC	88776655	C++
1	ABC	88776655	java
2	MNO	77889966	java
2	MNO	77889966	python
3	pqr	44556677	.NET

If we think about above table it is in first normal form because satisfy all rule specify as per 1NF
But the limitation is data is duplicated

Second Normal Form

Rules of Second normal Form

1. Table must be present in First Normal
2. Table should not use partial dependency

Note: Before partial dependency we should have to know what dependency is

Q. what is dependency?

Dependency means if particular column or attribute is responsible for identify unique record of database table called as dependency normally dependency manage by primary key

eid	name	salary
1	abc	10000
2	mno	20000
3	pqr	3000
4	abc	50000

if we think about above table suppose consider we want to identify the salary of employee whose name is abc but the problem we have two employee whose name is abc then we cannot identify the proper salary of employee means suppose we want to find abc whose salary is 50000 and if we try to identify by using name it is not possible because name is duplicated but we have one primary key column name as eid and eid is unique so using eid identify particular employee uniquely if we consider eid 4 we can say the name of employee is abc and salary is 50000 and if we consider eid 1 we can say name of employee is abc and salary is 10000 means we can identify uniquely employee record by using eid means employee record is dependent on eid column means we can single column represent complete row called as dependency in

Q. what is partial dependency?

when table has primary key that made up to two or more columns , then all the columns(not included in primary key) in that table should depend on the entire primary key and not on part of it. If any column (which is not in the primary key) depends on a part of primary key then we say we have partial dependency in the table.

Example: Suppose we have three tables

1. Student
2. Subject
3. Score

Student

student_id	student_name	branch
1	ABC	CSE
2	MNO	Mechanical
3	PQR	CSE

Subject

subject_id	subject_name
1	C
2	DSA
3	C++

score

student_id (fk)	subject_id (fk)	marks	teacher_name
1	1	70	Mr.CTeacher
1	2	80	DSATeacher
2	1	65	Mr.Ceacher

If we think about above tables we can we have score table which is responsible for maintain partial dependency because score table contain student_id and subject_id which work as foreign key in score table but indirectly they are primary key somewhere means we can say we have two primary keys in score table and we have two non-primary key column in score table one is marks and one is teacher_name and if we think about marks column it is dependent on both primary key or complete set of primary key.

Example: if we think 70 marks then we need to know the student name as well as subject name for this marks so if we want to identify student we need to know the student_id and if want to know the subject name we need to know the subject_id means marks column is dependent all primarys keys in table so we can say marks column not generate the partial dependency but if we think about teacher column so if we want to identity the teacher we just know subject_id we not required to know the student_id means we have two primary keys in score table but teacher_name is dependent on half primary key or not dependent set of primary key so here we can san teacher_name column is responsible for generate the partial dependency so this is major we can say score table not in second normal form

so if we want to score table should be second normal form we need to remove partial dependency from score table means you can remove the teacher column from score table and add teacher column subject table if your assign single subject for single teacher shown in following diagram.

Student

student_id	student_name	branch
1	ABC	CSE
2	MNO	Mechanical
3	PQR	CSE

Subject

subject_id	subject_name	teacher
1	C	CTeacher
2	DSA	DSTeacher
3	C++	CPPTeacher

score

student_id (fk)	subject_id (fk)	<u>marks</u>
1	1	70
1	2	80
2	1	65

But if you assign multiple subject to single teacher or single subject to multiple teacher then you required to create separate table teacher and separate table for subject and one intermediate table which contain only foreign keys which represent the relation between subject and teacher shown in following diagram.

Student

student_id	student_name	branch
1	ABC	CSE
2	MNO	Mechanical
3	PQR	CSE

Subject

subject_id	subject_name
1	C
2	DSA
3	C++

teacher_id	teacher_name
1	ABC
2	MNO

score

student_id (fk)	subject_id (fk)	<u>marks</u>
1	1	70
1	2	80
2	1	65

subject_id	teacher_id
1	1
1	2
2	1

Third Normal Form

Rules of Third Normal Form

1. Table should present in 2nd normal form
2. Table should not use Transitive dependency

Q. What is transitive dependency?

If any non primary key column is dependent on any other non primary key column called as transitive dependency.

Suppose consider we have score table from previous example and we add two columns in score table name as exam_type and total_marks

Student			Subject		
student_id	student_name	branch	subject_id	subject_name	teacher_name
1	ABC	CSE	1	C	CTeacher
2	MNO	Mechanical	2	DSA	DSTeacher
3	PQR	CSE	3	C++	CPPTeacher

score				
student_id (fk)	subject_id (fk)	obt marks	exam_type	total_marks
1	1	70	theory	100
1	2	80	theory	100
2	1	65	practical	80

If we think about above database we can say we have score table which contain two primary key column i.e. foreign key and three non primary key columns obt marks, exam_type and total_marks if we think about obt marks column it is dependent on subject_id and student_id we already discuss in 2nd normal form but here if we think about total_marks column it is dependent on exam_type suppose consider we say total marks is 100 so the question for which exam so for that we need to know the exam type here exam_type is out non primary key column so here we say total_marks is non primary key and exam_type is also non primary key column so as per the rule of transitive dependency if any non primary key column is dependent on any other non-primary key column called as transitive dependency

Means here we can say our Score table not in 3rd normal form

if we want to arrange score table in 3rd normal form we have to remove exam_type and total_marks from score table and create one more table in database as exam with three column i.e. exam_id, exam_type and total_marks and mark exam_id as primary key column in exam table and pass its reference in score table so you can identify obtain of student with identity subject id, and exam id shown in following diagram.

Student			Subject		
student_id	student_name	branch	subject_id	subject_name	teacher_name
1	ABC	CSE	1	C	CTeacher
2	MNO	Mechanical	2	DSA	DSTeacher
3	PQR	CSE	3	C++	CPPTeacher

score				exam_id	examp_type	total_marks
student_id (fk)	subject_id (fk)	obt marks	exam_id	1	theory	100
1	1	70	1	2	practical	80
1	2	80	1			
2	1	65	2			

Fourth Normal Form

- A relation will be in 4NF if it is in Boyce Codd normal form and has no multi-valued dependency.
- For a dependency $A \twoheadrightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no transition dependency exists.
BCNF	A stronger definition of 3NF is known as Boyce Codd's normal form.
4NF	A relation will be in 4NF if it is in Boyce Codd's normal form and has no multi-valued dependency.
5NF	A relation is in 5NF. If it is in 4NF and does not contain any join dependency, joining should be lossless.

Advantages of Normalization

- Normalization helps to minimize data redundancy.
- Greater overall database organization.
- Data consistency within the database.

- Much more flexible database design.
- Enforces the concept of relational integrity.

Disadvantages of Normalization

- You cannot start building the database before knowing what the user needs.
- The performance degrades when normalizing the relations to higher normal forms, i.e., 4NF, 5NF.
- It is very time-consuming and difficult to normalize relations of a higher degree.
- Careless decomposition may lead to a bad database design, leading to serious problems.

Sub Query /Nested Query

Query within Query called as sub Query

A Sub Query may be occurring in:

1. A select clause
2. A From clause
3. Where clause

A sub Query can be nested inside a select, insert, update or delete statement or inside any another sub query.

A Sub Query is usually added within the where clause of another SQL Select Statement

You can use the comparison operators such as < > or = etc. The comparison operator can also be multiple row operators such as IN, Any, All, exists etc. Means we required to use conjunction operator in sub query

There are two types of sub query

1. Inner query or nested query
2. Co-related sub query

Syntax:

Select *from outerquery where conjection operator (select *from inner query);

Important points related with sub query

1. A SubQuery (inner query) executes once before the main query (outer query)
2. The main query (outer query) uses the sub query result.

Example: WA SQL Query to find the second highest salary of employee using sub query

```
mysql> select *from employee order by salary desc;
```

empcode	name	contact	salary
115	abcd	abcd@gmail.com	6566666
100	Rajdeep	444444444444	100000
101	sam	4567	100000
501	Rajdeep	2323322	100000
130	sanjay	sanjay@gmail.com	70000
103	ram	1235455	60000
2000	raj	raj@gmail.com	60000
102	shyam	123545	50000
105	rajesh	4543221	50000
110	avinash	44444	40000
1999	ram	777777777	40000
104	dinesh	343434	NULL
111	ran	456447	NULL

13 rows in set (0.04 sec)

```
mysql> select max(salary) from employee where salary<(select max(salary) from employee);
```

max(salary)
100000

1 row in set (0.03 sec)

~~select max(salary) from employee where salary<6566666;~~

100000

excute inner query first and its
result provide as input to outer query

Note: when your sub query return single result or inner query return single result then can use the relational operator as conjection between outer query and inner query and when your sub query return more than one result or multiple results then you can use IN , any ,exists, All etc. operator as conjection operator between outer query and inner query

```
mysql> create table project(pid int(5) primary key auto_increment,name varchar(200) not null ,location varchar(200) not null);
```

Query OK, 0 rows affected, 1 warning (0.09 sec)

```
mysql> create table employee(eid int(5) primary key auto_increment,name varchar(200) not null, email varchar(200) not null unique, contact varchar(20) not null unique, salary int(5) ,pid int(5),foreign key(pid) references Project(pid));
```

Query OK, 0 rows affected, 3 warnings (0.10 sec)

```
mysql>
```

Q. WA SQL Query to find the employee list whose is working on LMS project.

Select * from employee where pid=(select pid from project where pname='LMS')

Project

pid	pname	plocation
1	LMS	DELHI
2	GMS	MUMBAI
3	HMS	PUNE

Employee

eid	name	email	contact	salary	pid
1	ram	ram@gmail.com	55443	10000	1
2	shyam	shyam@	33442222	20000	1
3	ganesh	ganesh@	11111111	30000	2
4	rajesh	rajesh@	2223333	40000	2
5	dinesh	dinesh@	6666666	50000	1

select pid from project where pname='LMS';

1

select *from employee where pid=(select pid from project where pname='LMS';);

select *from employee where pid=1;

Select * from employee where pid=(select pid from project where pname='LMS');

```
mysql> select *from employee where pid=(select pid from project where name='LMS');
```

eid	name	email	contact	salary	pid
1	ram	ram@gmail.com	123456789	10000	1
2	shyam	shyam@gmail.com	123456733	20000	1
5	rajesh	rajesh@gmail.com	623456733	30000	1

3 rows in set (0.02 sec)

Example: WA SQL Query to display the employee list who is working on at least one project

Select * from employee where pid in(select pid from project);

Select * from employee where pid in(select pid from project);

```
mysql> select *from employee where pid IN(select pid from project);
```

eid	name	email	contact	salary	pid
1	ram	ram@gmail.com	123456789	10000	1
2	shyam	shyam@gmail.com	123456733	20000	1
5	rajesh	rajesh@gmail.com	623456733	30000	1
3	ganesh	ganesh@gmail.com	823456733	20000	2
4	dinesh	dinesh@gmail.com	723456733	30000	2

5 rows in set (0.03 sec)

Co-related Sub query

<https://www.geeksforgeeks.org/nested-queries-in-sql/?ref=lbp>

Refer above links

if we use the inner query then first inner query get executed and generate the output and output of inner query provide as input to outer query but if we think about co –related sub query first outer query get executed and outer query reference use in inner query

Correlated subqueries are used for row-by-row processing. Each subquery is executed once for every row of the outer query.

A correlated subquery is evaluated once for each row processed by the parent statement. The parent statement can be a SELECT, UPDATE, or DELETE statement.

Syntax:

```

SELECT column1, column2, ....
FROM table1 outer
WHERE column1 operator
      (SELECT column1, column2
       FROM table2
       WHERE expr1 =
           outer.expr2);

```

A correlated subquery is one way of reading every row in a table and comparing values in each row against related data. It is used whenever a subquery must return a different result or set of results for each candidate row considered by the main query. In other words, you can use a correlated subquery to answer a multipart question whose answer depends on the value in each row processed by the parent statement.

Other use of correlation is in **UPDATE** and **DELETE**

CORRELATED UPDATE :

```

UPDATE table1 alias1
SET column = (SELECT expression
              FROM table2 alias2
              WHERE alias1.column =
                  alias2.column);

```

Use a correlated subquery to update rows in one table based on rows from another table.

CORRELATED DELETE :

```

DELETE FROM table1 alias1
WHERE column1 operator
      (SELECT expression
       FROM table2 alias2
       WHERE alias1.column = alias2.column);

```

Use a correlated subquery to delete rows in one table based on the rows from another table.

Example: WA SQL Query to find the employee who is working on project.

Select e.name, e.email, e.contact, e.salary from employee e where exists(select pid from project p where e.pid=p.pid);

```
mysql> select e.name,e.email,e.contact,e.salary from employee e where exists(select pid from project p where e.pid=p.pid);
```

```
mysql> select *from project;
```

pid	name	location
1	LMS	DELHI
2	GMS	MUMBAI
3	HMS	PUNE

3 rows in set (0.00 sec)

```
mysql> select *from employee;
```

eid	name	email	contact	salary	pid
1	ram	ram@gmail.com	123456789	10000	1
2	shyam	shyam@gmail.com	123456733	20000	1
3	ganesh	ganesh@gmail.com	823456733	20000	2
4	dinesh	dinesh@gmail.com	723456733	30000	2
5	rajesh	rajesh@gmail.com	623456733	30000	1

5 rows in set (0.00 sec)

How to use the Sub Query with delete Statement

pid	name	location
1	LMS	DELHI
2	GMS	MUMBAI
3	HMS	PUNE

3 rows in set (0.02 sec)

```
mysql> select *from employee;
```

eid	name	email	contact	salary	pid
1	ram	ram@gmail.com	123456789	10000	1
2	shyam	shyam@gmail.com	123456733	20000	1
3	ganesh	ganesh@gmail.com	823456733	20000	2
4	dinesh	dinesh@gmail.com	723456733	30000	2
5	rajesh	rajesh@gmail.com	623456733	30000	1

5 rows in set (0.00 sec)

Example: delete all employees who are working on GMS project by using sub query.

```
mysql> delete from employee where pid=(select pid from project where name='GMS');  
Query OK, 2 rows affected (0.04 sec)
```

```
mysql> select *from employee;
```

eid	name	email	contact	salary	pid
1	ram	ram@gmail.com	123456789	10000	1
2	shyam	shyam@gmail.com	123456733	20000	1
5	rajesh	rajesh@gmail.com	623456733	30000	1

3 rows in set (0.00 sec)

SubQuery with update statement

Example: WA SQL Query to increase salary by 30% of all employees who are working on LMS project using SubQuery.

```
mysql> update employee set salary=salary+(salary*30/100) where pid=(select pid from project where name='LMS');
Query OK, 3 rows affected (0.01 sec)
Rows matched: 3  Changed: 3  Warnings: 0

mysql> select *from employee;
+----+-----+-----+-----+-----+-----+
| eid | name  | email          | contact | salary | pid |
+----+-----+-----+-----+-----+-----+
| 1   | ram   | ram@gmail.com  | 123456789 | 13000 | 1   |
| 2   | shyam | shyam@gmail.com | 123456733 | 26000 | 1   |
| 5   | rajesh | rajesh@gmail.com | 623456733 | 39000 | 1   |
+----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

The SQL EXISTS Operator

The EXISTS operator is used to test for the existence of any record in a subquery.

The EXISTS operator returns TRUE if the subquery returns one or more records

Q. what is diff between IN and exists operator?

IN	Exists
IN can be used as a replacement for multiple OR conditions	To determine if any values are returned or not we use exists
Faster execution as SubQuery execute only once	Slower execution as Exists operator forces the sub-query to execute again and again for each Subsequence outer query
In the IN condition SQL Engine compares all the values in IN clauses	Once true is evaluated in the exists condition then SQL Engine will stop the process for further matching
Follow bottom up approach	Follow TOP-DOWN approach for execution
IN Operator cannot compare any within null value	The exists operator can compare with null value
A direct set of values can be given for comparison	Cannot compare direct the values subquery need to given

Example: WA SQL Query to display the employee list who are not working on any project.

```
mysql> select *from employee e where not exists(select p.pid from project p where p.pid=e.pid);
+----+-----+-----+-----+-----+-----+
| eid | name  | email          | contact | salary | pid |
+----+-----+-----+-----+-----+-----+
| 6   | sagar | sagar@gmail.com | 98765678 | 40000 | NULL |
+----+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

1. WA SQL Query to display the project list which have any employees.

Select * from project p where exists(select p.pid from employee e where e.pid=p.pid);

2. WA SQL Query to display the project list which don't have any employees.

```
mysql> select * from project p where exists(select p.pid from employees e where e.pid=p.pid);
+----+-----+-----+
| pid | name | location |
+----+-----+-----+
| 1   | LMS  | DELHI    |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> select * from project p where not exists(select p.pid from employees e where e.pid=p.pid);
+----+-----+-----+
| pid | name | location |
+----+-----+-----+
| 2   | GMS  | MUMBAI   |
| 3   | HMS  | PUNE     |
| 4   | DMS  | CHENNAI  |
| 5   | BMS  | NAGPUR   |
+----+-----+-----+
4 rows in set (0.02 sec)
```

Difference between Nested Query, Correlated Query, and Join Operation

Parameters	Nested Query	Correlated Query	Join Operation
Definition	In Nested query, a query is written inside another query and the result of the inner query is used in the execution of the outer query.	In Correlated query, a query is nested inside another query and an inner query uses values from the outer query.	The join operation is used to combine data or rows from two or more tables based on a common field between them. INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL JOIN are different types of Joins.
Approach	Bottom-up approach i.e. Inner query runs first, and only once. The outer query is executed with result from the Inner query.	Top to Down Approach i.e. Outer query executes first and for every Outer query row Inner query is executed.	It is basically cross product satisfying a condition.
Dependency	Inner query execution is not dependent on Outer query.	Inner query is dependent on the Outer query.	There is no Inner Query or Outer Query. Hence, no dependency is there.
Performance	Performs better than Correlated Query but is slower than Join Operation.	Performs slower than both Nested Query and Join operations as for every outer query inner query is executed	By using joins we maximize the calculation burden on the database but joins are better optimized by the server so the retrieval time of the query using joins will almost always be faster than that of a subquery.

Views in SQL

Q. What is view?

View in SQL is kind of virtual tables. A View also has row and column as they are in real table.

In database you can create view by selecting field from one or more tables present in the database. A View can have all column from another table or specific column from another table.

Q. Why use view or what is benefit of Views?

- 1. Restricting data access:** View Provide an additional level of table security by restricting access to predetermined set of rows and column of table.
- 2. Hiding data complexity:** A view can hide complexity that exists in multiple table join.
- 3. Rename columns:** View can also be used to rename the column without affecting the base tables provided the number of columns in view must match the number of column specified in select statement
- 4. Multiple View Facility:** you can apply multiple views on single table.

How to Create View in SQL

Syntax: create view viewname as select column1, column2....column...n from tablename;

Example:

```
mysql> select *from employee;
```

eid	name	email	contact	salary	pid
1	ram	ram@gmail.com	123456789	13000	1
2	shyam	shyam@gmail.com	123456733	26000	1
5	rajesh	rajesh@gmail.com	623456733	39000	1
6	sagar	sagar@gmail.com	98765678	40000	NULL

```
4 rows in set (0.01 sec)
```

Suppose consider we have table name as Employee and we want to fetch name and contact from employee using view means we want to provide access of only two column of employee table.

Create view empview as select name,contact from employee

```
mysql> create view empview as select name,contact from employee;
Query OK, 0 rows affected (0.07 sec)

mysql> select *from empview;
```

name	contact
ram	123456789
shyam	123456733
rajesh	623456733
sagar	98765678

```
4 rows in set (0.01 sec)
```

Create view empview as select name, contact from employee;

You can use join with view

```
mysql> create view empjoin as select p.name as 'Project Name',e.name as 'Employee Name', e.salary as 'Employee Salary' from
Employee e inner join Project p on e.pid=p.pid;
Query OK, 0 rows affected (0.04 sec)

mysql> select *from empjoin;
+-----+-----+-----+
| Project Name | Employee Name | Employee Salary |
+-----+-----+-----+
| LMS          | ram           | 13000           |
| LMS          | shyam        | 26000           |
| LMS          | rajesh       | 39000           |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

How to drop the view or How to delete the view

If we want to drop view or delete view we have following command

Syntax: drop view viewname;

```
mysql> drop view empjoin;
Query OK, 0 rows affected (0.04 sec)

mysql> select *from empjoin;
ERROR 1146 (42S02): Table 'mayjuneaug.empjoin' doesn't exist
mysql>
```

dropping view

after drop view.

Update view

There are certain conditions needed to be satisfied to update a view. If any one of these conditions is not met, then we will not be allowed to update the view.

- The SELECT statement which is used to create the view should not include GROUP BY clause or ORDER BY clause.
- The SELECT statement should not have the DISTINCT keyword.
- The View should have all NOT NULL values.
- The view should not be created using nested queries or complex queries.
- The view should be created from a single table. If the view is created using multiple tables then we will not be allowed to update the view.

If we want to modify view or update view we have to use following command.

Syntax:

Create or replace view viewname as select column1, column2.....column...n from tablename;

```
mysql> create view empjoin as select name,contact from employee;
Query OK, 0 rows affected (0.01 sec)

mysql> select *from empjoin;
+-----+-----+
| name | contact |
+-----+-----+
| ram  | 123456789 |
| shyam | 123456733 |
| rajesh | 623456733 |
| sagar | 98765678 |
+-----+-----+
4 rows in set (0.00 sec)

mysql>
```

If we think about above screen shot we create view with column name and contact and we want to update the view and add salary column in view name as empjoin shown in following screen shot

```
mysql> create or replace view empjoin as select name,contact,salary from employee;
Query OK, 0 rows affected (0.02 sec)

mysql> select *from empjoin;
+-----+-----+-----+
| name | contact | salary |
+-----+-----+-----+
| ram  | 123456789 | 13000 |
| shyam | 123456733 | 26000 |
| rajesh | 623456733 | 39000 |
| sagar | 98765678 | 40000 |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> delete from empjoin where name='ram';
Query OK, 1 row affected (0.03 sec)

mysql> select *from employee;
+-----+-----+-----+-----+-----+-----+
| eid | name | email | contact | salary | pid |
+-----+-----+-----+-----+-----+-----+
| 2 | shyam | shyam@gmail.com | 123456733 | 26000 | 1 |
| 5 | rajesh | rajesh@gmail.com | 623456733 | 39000 | 1 |
| 6 | sagar | sagar@gmail.com | 98765678 | 40000 | NULL |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> update empjoin set salary=100000 where name='shyam';
Query OK, 1 row affected (0.04 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> select *from employee;
+-----+-----+-----+-----+-----+-----+
| eid | name | email | contact | salary | pid |
+-----+-----+-----+-----+-----+-----+
| 2 | shyam | shyam@gmail.com | 123456733 | 100000 | 1 |
| 5 | rajesh | rajesh@gmail.com | 623456733 | 39000 | 1 |
| 6 | sagar | sagar@gmail.com | 98765678 | 40000 | NULL |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> _
```

```
mysql> update projectDetails set ename='mahesh' where salary=16900;
Query OK, 1 row affected (0.13 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from projectDetails;
+-----+-----+-----+-----+
| ename | pname | location | salary |
+-----+-----+-----+-----+
| mahesh | LMS   | DELHI   | 16900  |
| shyam  | LMS   | DELHI   | 33800  |
| dinesh  | LMS   | DELHI   | 84500  |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Note: if we perform any operation on view it is indirectly perform on associated table.

Insert into views

In SQL, you cannot directly insert records into a view if it's based on multiple tables unless it's an "updatable view" and meets certain criteria. An updatable view means that you can perform DML (Data Manipulation Language) operations such as INSERT, UPDATE, and DELETE on the view.

Check if the View is Updatable: You can determine if a view is updatable by examining its definition and ensuring it meets certain criteria. For example, if the view contains columns from only one table, it's more likely to be updatable. Views involving multiple tables may not be updatable unless they meet specific conditions like having a primary key defined, etc.

Create Instead of Triggers: If the view is not inherently updatable, you can use instead-of triggers to handle inserts, updates, and deletes. When you insert a record into the view, the trigger fires and you can define the logic to handle the insertion across multiple tables.

Stored Procedure or Procedure

Q. what is procedure?

Procedure is block of SQL statement that you can reuse it multiple times like as function in c language. Means some time if we have some logic in SQL and if we want to reuse it multiple then you can write all logic in procedure and you can reuse the procedure multiple times

What is a Stored Procedure?

A stored procedure is a prepared SQL code that you can save, so the code can be reused over and over again. So if you have an SQL query that you write over and over again, save it as a stored procedure, and then just call it to execute it.

You can also pass parameters to a stored procedure, so that the stored procedure can act based on the parameter value(s) that is passed.

How to work with procedure by using MYSQL

If we want to work with procedure in MYSQL we have to use the following syntax.

Syntax:

delimiter character

```
create procedure procedurename(IN variablename datatype(size),OUT variablename datatype(size))
```

```
begin
```

```
    write here your logics
```

```
end
```

```
character ;
```

Example:

```
delimiter $$
```

```
create procedure test(IN name varchar(200),email varchar(200),contact varchar(200))
```

```
begin
```

```
insert into employee values(name, email, contact)
```

```
end
```

```
$$
```

If we think about above example we have statement

delimiter \$\$ here delimiter indicate ending of procedure or ending SQL statement and it is by default semi colon but when we use the procedure or trigger etc. then you need to set different character as delimiter because in procedure we have other logical statement and we need to ends with semi colon then if want to identity the procedure end so you need to different delimiter character

Create procedure test(IN name varchar(200),email varchar(200),contact varchar(200))

If we think about above statement we have create procedure indicate create procedure or it is keyword for creating procedure test is your procedure name, name, email and contact are the parameters to procedure

There are three types of parameter in procedure

IN: this parameter just accepts input as parameter and by default every parameter consider as IN parameter if we not use IN keyword with parameter.

OUT: OUT parameter is used for when we want to take some result from procedure definition to procedure calling then we can use OUT parameter.

IN OUT: IN OUT parameter means single parameter can work IN put parameter as well as output parameter.

begin keyword indicate you can writing logic of procedure and **end** keyword indicate you can end the procedure it is like as opening curly and closing curly of function block.

Once we create procedure and if we want to reuse the procedure code you have to call it.

Syntax: call procedurename (parameters)

Example: call test ('abc','abc@gmail.com','123456');

Example: Create Procedure name as save Project and insert record in project table by using procedure.

```
mysql> delimiter $$
mysql> create procedure saveProject(n varchar(200),l varchar(200))
-> begin
-> insert into project values('0',n,l);
-> end
-> $$
Query OK, 0 rows affected (0.08 sec)
```

Once we create procedure you can reuse the procedure and if we want to reuse the procedure you have to call it for that we have the following syntax

Syntax: call procedurename (parameters);

Example: call saveProject ('BMS','Nagpur');

When we call the procedure the SQL engine execute or call the definition of procedure and execute the procedure.

```
mysql> call saveProject('BMS','nagpur')$$
Query OK, 1 row affected (0.06 sec)
```

```
mysql> delimiter ;
mysql> call saveProject('HMS','PUNE');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select *from project;
```

pid	name	location
1	LMS	DELHI
2	MLS	PUNE
3	GMS	Mumbai
4	BMS	nagpur
5	HMS	PUNE

```
5 rows in set (0.00 sec)
```

```
mysql>
```

Example: Create procedure name as saveCourse (coursename varchar (200)) and store course data in course table by using procedure.

```
mysql> delimiter //
mysql> create procedure saveCourse(cname varchar(200))
    -> begin
    -> insert into course values('0',cname);
    -> end
    -> //
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> call saveCourse('HADOOP');
Query OK, 1 row affected (0.01 sec)

mysql> call saveCourse('PYTHON');
Query OK, 1 row affected (0.01 sec)
```

How to use out parameter of procedure

Basically out parameter is used for when we want to get some resultant value from procedure definition at procedure calling point then we can use out parameter.

Example: we want to design procedure pass project name to project table and return its location so here you have to set project name as IN parameter and location as OUT parameter.

Step to work using out parameter

1) Define procedure using out parameter

Example:

```
mysql> delimiter //
mysql> create procedure getLocation(IN cname varchar(200) ,OUT ploc varchar(200))
    -> begin
```

2) Store data in OUT parameter by using INTO keyword

INTO keyword help us to store output data in OUT parameter shown in following code.

Example:

```
mysql> delimiter //
mysql> create procedure getLocation(IN cname varchar(200) ,OUT ploc varchar(200))
    -> begin
    -> select location INTO ploc from project where name=cname;
    -> end
    -> //
Query OK, 0 rows affected (0.03 sec)
```

If we think about above select statement you can select location INTO ploc means we fetch location column from project table and store in ploc OUT variable using INTO keyword means location value stored in ploc variable which is output parameter variable in our procedure

3) Call the procedure and store out parameter value in variable at procedure calling

If we want to declare variable in MYSQL we have to use @symbol in MYSQL

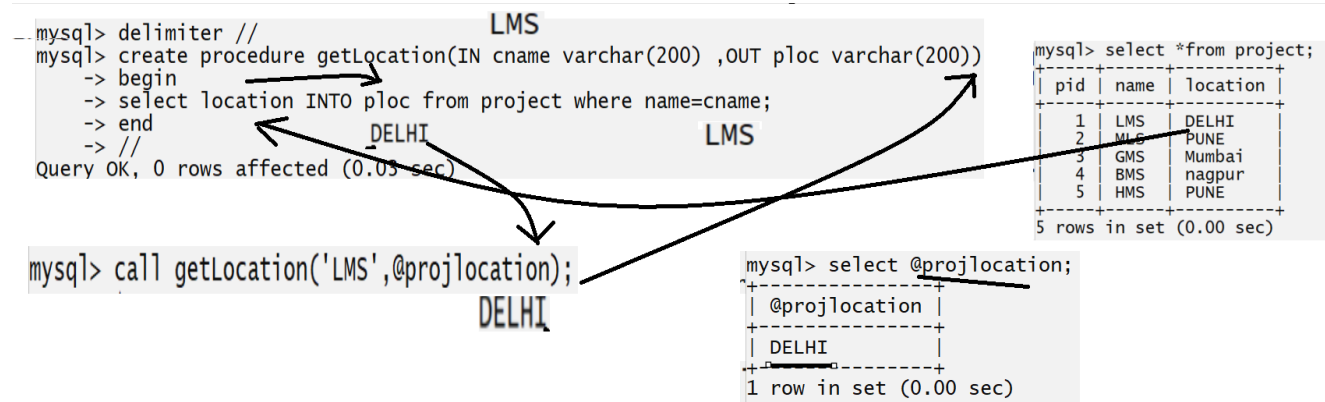
```
mysql> call getLocation('LMS',@projlocation);
Query OK, 1 row affected (0.02 sec)
```

If we think about above screen short we have LMS is our IN parameter value and @projlocation it is our output parameter means this parameter able to hold the value of OUTPUT parameter which we mention in procedure definition.

4) Print the out parameter using select command

```
mysql> select @projlocation;
+-----+
| @projlocation |
+-----+
| DELHI         |
+-----+
1 row in set (0.00 sec)
```

Following screen short show the complete flow of output parameter.



Example: WA Procedure to pass course id as input parameter and get course name as output parameter.

```
mysql> delimiter //
mysql> create procedure getCourse(IN courseid int(5),OUT coursename varchar(200))
-> begin
-> select name INTO coursename from course where cid=courseid;
-> end
-> //
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> call getCourse(1,@coursename);
-> //
Query OK, 1 row affected (0.00 sec)

mysql> select @coursename//
+-----+
| @coursename |
+-----+
| JAVA        |
+-----+
1 row in set (0.00 sec)
```

Example: Create Procedure to find the second highest salary of employee.

```
mysql> delimiter //
mysql> create procedure getSecondMax(OUT smax int(5))
    -> begin
    -> select max(salary) from employee where salary < (select max(salary) from employee);
    -> end
    -> //
Query OK, 0 rows affected, 1 warning (0.04 sec)

mysql> call getSecondMax(@secondhighest);
    -> //
+-----+
| max(salary) |
+-----+
|          50000 |
+-----+
1 row in set (0.00 sec)
```

Trigger in MYSQL

Q. What is trigger?

Trigger is same like as procedure in MYSQL and which is used for write some block level logics. The major difference between trigger and procedure is procedure need to call manually and trigger call or execute automatically on specified event by database engine.

Example: trigger call on DML event like as after inserting record in particular table or deleting record from particular etc.

How to Create Trigger in MYSQL

If we want to create trigger in MYSQL we have following syntax.

delimiter character

create trigger triggername [before | after] [insert | delete | update | etc] on tablename
for each row

write here your trigger logics

end

character

Example: delimiter \$\$

create trigger testtrig before insert on account

for each row

write here your logics

end

\$\$

There are six types of trigger in MYSQL

Before insert: this type of trigger is executed before inserting operation.

After insert: this type of trigger is executed after inserting operation

Before Delete: this type of trigger execute before deletion operation.

After Delete: this type of trigger execute after deletion operation

Before Update: execute trigger before update operation

After Update: execute trigger after update operation.

Example: we want to create ministatement record automatically when we update the account table

```
mysql> create table account(accno int(5) primary key auto_increment,name varchar(200) not null , balace int(5));
Query OK, 0 rows affected, 2 warnings (0.15 sec)
```

```
mysql> insert into account values(101,'ABC',10000);
Query OK, 1 row affected (0.02 sec)
```

```
mysql> insert into account values(102,'MNO',20000);
Query OK, 1 row affected (0.03 sec)
```

```
mysql> select *from account;
```

accno	name	balace
101	ABC	10000
102	MNO	20000

```
mysql> create table ministatement(accnot int(5),prevbalance int(5),newbalance int(5),transdate date);
Query OK, 0 rows affected, 3 warnings (0.07 sec)
```

```
mysql> delimiter $$
```

```
mysql> create trigger createministatement before update on account
```

```
-> for each row
```

```
-> insert into ministatement values(old.accno,old.balace,new.balace,curDate());
```

```
-> end
```

```
-> $$
```

```
Query OK, 0 rows affected (0.04 sec)
```

//index,cursor,for loop,if else,keyws

Q1. What is the difference between SQL and MySQL?

SQL	MySQL
SQL is a standard language which stands for Structured Query Language based on the English language	MySQL is a database management system.
SQL is the core of the relational database which is used for accessing and managing database	MySQL is an RDMS (Relational Database Management System) such as SQL Server, Informix etc.

Q. What are some common clauses used with SELECT query in SQL?

The following are some frequent SQL clauses used in conjunction with a SELECT query:

WHERE clause: In SQL, the WHERE clause is used to filter records that are required depending on certain criteria.

ORDER BY clause: The ORDER BY clause in SQL is used to sort data in ascending (ASC) or descending (DESC) order depending on specified field(s) (DESC).

GROUP BY clause: GROUP BY clause in SQL is used to group entries with identical data and may be used with aggregation methods to obtain summarised database results.

HAVING clause in SQL is used to filter records in combination with the GROUP BY clause. It is different from WHERE, since the WHERE clause cannot filter aggregated records.

Q. What are UNION, MINUS and INTERSECT commands?

The UNION operator is used to combine the results of two tables while also removing duplicate entries.

The MINUS operator is used to return rows from the first query but not from the second query.

The INTERSECT operator is used to combine the results of both queries into a single row.

Before running either of the above SQL statements, certain requirements must be satisfied –

Within the clause, each SELECT query must have the same amount of columns.

The data types in the columns must also be comparable.

In each SELECT statement, the columns must be in the same order.

Q. List the different types of relationships in SQL.

There are different types of relations in the database:

One-to-One – This is a connection between two tables in which each record in one table corresponds to the maximum of one record in the other.

One-to-Many and Many-to-One – This is the most frequent connection, in which a record in one table is linked to several records in another.

Many-to-Many – This is used when defining a relationship that requires several instances on each sides.

Self-Referencing Relationships – When a table has to declare a connection with itself, this is the method to employ.

Q12. What is OLTP?

OLTP, or online transactional processing, allows huge groups of people to execute massive amounts of database transactions in real time, usually via the internet. A database transaction occurs when data in a database is changed, inserted, deleted, or queried.

Q13. What are the differences between OLTP and OLAP?

OLTP stands for online transaction processing, whereas OLAP stands for online analytical processing.

OLTP is an online database modification system, whereas OLAP is an online database query response system.

Q. NoSQL vs SQL

In summary, the following are the five major distinctions between SQL and NoSQL:

Relational databases are SQL, while non-relational databases are NoSQL.

SQL databases have a specified schema and employ structured query language. For unstructured data, NoSQL databases use dynamic schemas.

SQL databases scale vertically, but NoSQL databases scale horizontally.

NoSQL databases are document, key-value, graph, or wide-column stores, whereas SQL databases are table-based.

SQL databases excel in multi-row transactions, while NoSQL excels at unstructured data such as documents and JSON

Q49. What is the difference between DELETE and TRUNCATE statements?

DELETE	TRUNCATE
Delete command is used to delete a row in a table.	Truncate is used to delete all the rows from a table.
You can rollback data after using delete statement.	You cannot rollback data.
It is a DML command.	It is a DDL command.
It is slower than truncate statement.	It is faster.

Q. What is the difference between clustered and non-clustered index in SQL?

The differences between the clustered and non clustered index in SQL are :

1. Clustered index is used for easy retrieval of data from the database and its faster whereas reading from non clustered index is relatively slower.
2. Clustered index alters the way records are stored in a database as it sorts out rows by the column which is set to be clustered index whereas in a non clustered index, it does not alter the way it was stored but it creates a separate object within a table which points back to the original table rows after searching.
3. One table can only have one clustered index whereas it can have many non clustered index.

Q. What are Entities and Relationships?

Entities: A person, place, or thing in the real world about which data can be stored in a database. Tables store data that represents one type of entity. For example – A bank database has a customer table to store customer information. The customer table stores this information as a set of attributes (columns within the table) for each customer.

Relationships: Relation or links between entities that have something to do with each other. For example – The customer name is related to the customer account number and contact information, which might be in the same table. There can also be relationships between separate tables (for example, customer to accounts).

Q.What is an Index?

An index refers to a performance tuning method of allowing faster retrieval of records from the table.

An index creates an entry for each value and hence it will be faster to retrieve data.

Q. Explain different types of index in SQL.

There are three [types of index in SQL](#) namely:

Unique Index:

This index does not allow the field to have duplicate values if the column is unique indexed. If a primary key is defined, a unique index can be applied automatically.

Clustered Index:

This index reorders the physical order of the table and searches based on the basis of key values. Each table can only have one clustered index.

Non-Clustered Index:

Non-Clustered Index does not alter the physical order of the table and maintains a logical order of the data. Each table can have many nonclustered indexes.

Q. What is the ACID property in a database?

ACID stands for Atomicity, Consistency, Isolation, Durability. It is used to ensure that the data transactions are processed reliably in a database system.

- **Atomicity:** Atomicity refers to the transactions that are completely done or failed where transaction refers to a single logical operation of a data. It means if one part of any transaction fails, the entire transaction fails and the database state is left unchanged.
- **Consistency:** Consistency ensures that the data must meet all the validation rules. In simple words, you can say that your transaction never leaves the database without completing its state.
- **Isolation:** The main goal of isolation is concurrency control.
- **Durability:** Durability means that if a transaction has been committed, it will occur whatever may come in between such as power loss, crash or any sort of error.

Q. What is Auto Increment in SQL?

Autoincrement keyword allows the user to create a unique number to get generated whenever a new record is inserted into the table.

This keyword is usually required whenever PRIMARY KEY in SQL is used.

[AUTO INCREMENT keyword](#) can be used in Oracle and IDENTITY keyword can be used in SQL SERVER.

Q. What is a Datawarehouse?

Datawarehouse refers to a central repository of data where the data is assembled from multiple sources of information. Those data are consolidated, transformed and made available for the mining as well as online processing. Warehouse data also have a subset of data called Data Marts

